Machine Learning Fundamentals

COMSATS University Islamabad
Vehari Campus

Instructor:
Junaid Iqbal Baig

Contents

1 Lecture 01 **9**

2 Preliminary Concepts for Machine Learning **9**
2.1 Data, Variables, and Samples . . . 9
2.2 Random Variables and Probability . . . 10
2.3 Basic Probability Concepts . . . 10
2.3.1 Probability of an Event . . . 10
2.3.2 Joint Probability . . . 10
2.3.3 Conditional Probability . . . 10
2.4 Bayes' Theorem . . . 10
2.5 Independence and Conditional Independence . . . 11
2.6 Statistical Measures . . . 11
2.6.1 Mean (Arithmetic Mean) . . . 11
2.6.2 Geometric Mean . . . 11
2.6.3 Variance and Standard Deviation . . . 11
2.7 Probability Distributions in Machine Learning . . . 12
2.7.1 Bernoulli Distribution . . . 12
2.7.2 Gaussian Distribution . . . 12
2.8 Linear Algebra Essentials . . . 12
2.8.1 Vectors . . . 12
2.8.2 Dot Product . . . 12
2.8.3 Matrix Representation . . . 12
2.9 Functions and Models . . . 12
2.10 Loss Functions and Error Measurement . . . 13
2.11 Optimization and Learning . . . 13
2.12 Generalization and Overfitting . . . 13

3 Introduction to Machine Learning **13**
3.1 Definition . . . 13
3.2 Examples . . . 14
3.3 Applications of Machine Learning . . . 14

4 Comparison: AI, Machine Learning, and Deep Learning **14**
4.1 Scenarios . . . 15

5 Learning Paradigms **15**
5.1 1. Supervised Learning . . . 15
5.2 2. Unsupervised Learning . . . 15
5.3 3. Semi-Supervised Learning . . . 15
5.4 4. Reinforcement Learning . . . 15

6 Case Studies **15**
6.1 1. Playing Checkers . . . 15
6.2 2. Speech Recognition . . . 16
6.3 3. Handwriting Recognition . . . 16

7 Questions **16**

8 Lecture 02: Types of Learning **18**
8.1 Inductive Learning . . . 18
8.2 Deductive Learning . . . 19
8.3 Online (Incremental) Learning . . . 19
8.4 Batch (Offline) Learning . . . 20
8.5 Instance-Based Learning . . . 20
8.6 Model-Based Learning . . . 20
8.7 Instance-Based vs Model-Based (Technical View) . . . 21
8.8 Supervised Learning Basics: Classification vs Regression . . . 21
8.9 Popular Machine Learning Algorithms: Supervised vs Unsupervised . . . 22
8.10 K-Nearest Neighbors (k-NN): Distance Metrics and Example Problems . 22

9 Lecture 03: Decision Trees **25**
9.1 Entropy . . . 25
9.2 Information Gain (IG) . . . 25
9.3 Gini Index . . . 25
9.4 Dataset: Play Tennis . . . 26
9.5 Tree Structure . . . 26
9.6 Step 1: Entropy of Entire Dataset . . . 26
9.7 Step 2: Entropy for Each Feature . . . 27
9.7.1 Outlook . . . 27
9.7.2 Information Gain for Outlook . . . 27
9.7.3 Humidity . . . 27
9.7.4 Wind . . . 28
9.8 Step 3: Choose Best Feature for Root . . . 28
9.9 Step 4: Split Dataset and Repeat Recursively . . . 28

10 Random Forests **28**
10.1 Key Concepts . . . 28
10.2 Algorithm Steps . . . 29
10.3 Advantages of Random Forest . . . 29
10.4 Disadvantages . . . 30
10.5 Numerical Example: Random Forest on Play Tennis Dataset . . . 30
10.5.1 Step 1: Generate Bootstrap Samples . . . 30
10.5.2 Step 2: Build Decision Trees on Each Sample . . . 30
10.6 Tree Splits and Calculations . . . 30
10.6.1 Step 3: Make Predictions for a New Day . . . 31
10.6.2 Step 4: Final Prediction by Majority Vote . . . 31
10.6.3 Step 5: Advantages Observed . . . 31

11 Lecture 05 **32**

12 Linear Regression and Logistic Regression **32**
12.1 Linear Regression . . . 32
12.2 Example Dataset . . . 32
12.3 Step 1: Compute Means . . . 33
12.4 Step 2: Compute Slope . . . 33
12.5 Step 3: Compute Intercept . . . 33
12.6 Final Linear Regression Model . . . 34

12.7 Prediction Example . . . 34
12.8 Logistic Regression . . . 34
12.9 Sigmoid Function . . . 34
12.10 Numerical Example . . . 35
12.11 Learning Model Parameters using Gradient Descent . . . 36
12.12 Loss Function . . . 36
12.12.1 Training Procedure . . . 36
12.13 Numerical Example . . . 36
12.14 Visualization of Gradient Step . . . 37

13 Lecture 06 **38**

14 Introduction **38**

15 Bayes Theorem **38**

16 Naive Independence Assumption **38**

17 Training Dataset Example **39**

18 Step 1: Prior Probabilities **40**

19 Step 2: Conditional Probabilities **40**
19.1 Probability Table for Age . . . 40
19.2 Probability Table for Student . . . 40
19.3 Probability Table for Income . . . 40

20 Step 3: Classify a New Person **40**
20.1 Probability for Yes . . . 41
20.2 Probability for No . . . 41

21 Final Decision **41**

22 Visualization of Classification **41**

23 Laplace Smoothing **41**

24 Types of Naive Bayes **42**

25 Advantages **42**

26 Limitations **42**

27 Conclusion **42**

28 Lecture 07: ANN Training and Backpropagation **43**
28.1 Introduction . . . 43
28.2 ANN Architecture (Layers) . . . 43
28.3 Neuron Computation . . . 43
28.4 Activation Function (Sigmoid) . . . 44
28.5 Loss Function . . . 44

28.6 What is Backpropagation? . . . 44
28.7 Why Backpropagation is Needed . . . 44
28.8 Complete Numerical Example (2-Layer Network) . . . 44
28.8.1 Step 1: Forward Pass . . . 45
28.8.2 Step 2: Error . . . 45
28.8.3 Step 3: Backpropagation (Output Layer) . . . 45
28.8.4 Step 4: Update Output Weights . . . 45
28.8.5 Step 5: Backpropagation (Hidden Layer) . . . 45
28.8.6 Step 6: Update Hidden Weights . . . 46
28.9 Key Understanding . . . 46
28.10 Important Observations . . . 46

29 Lecture 08: Support Vector Machines (SVM) 47
29.1 Learning Objectives . . . 47
29.2 Introduction . . . 47
29.3 Hyperplane . . . 47
29.4 Maximum Margin . . . 47
29.5 Support Vectors . . . 47
29.6 Optimization Problem . . . 47
29.7 Step-by-Step Example . . . 48
29.8 Hard Margin vs Soft Margin . . . 48
29.9 Non-Linear Case . . . 48
29.10 Kernel Trick . . . 48
29.11 Types of Kernels . . . 49
29.12 Example Problem . . . 49
29.13 Step 1: Vector Representation . . . 49
29.14 Step 2: SVM Optimization Problem . . . 50
29.15 Step 3: Support Vectors . . . 50
29.16 Step 4: Weight Vector Form . . . 50
29.17 Step 5: Support Vector Conditions . . . 50
29.18 Step 6: Solve Simultaneous Equations . . . 51
29.19 Step 7: Final Weight Vector . . . 52
29.20 Step 8: Final Hyperplane . . . 52
29.21 Step 9: Margin . . . 52
29.21.1 Formula of Margin . . . 52
29.21.2 Step-by-step Calculation . . . 52
29.21.3 Final Margin . . . 52
29.22 Step 10: Prediction Rule . . . 52
29.23 Step 11: Drawing the Hyperplane . . . 53
29.23.1 Step 1: Convert to Slope-Intercept Form . . . 53
29.23.2 Step 2: Choose Two Points . . . 53
29.23.3 Step 3: Plot the Points . . . 53
29.24 Lagrange Multipliers (α_i) in SVM . . . 54
29.24.1 Definition . . . 54
29.24.2 Role of α_i . . . 54
29.24.3 Weight Vector in Dual Form . . . 54
29.24.4 Constraint on α_i . . . 54
29.24.5 Why α_i Cannot Be Found Easily . . . 55

29.25Practical Computation of Lagrange Multipliers (α) 55
29.25.1 Sequential Minimal Optimization (SMO) 55
29.25.2 Quadratic Programming (QP) Solvers 55
29.26Advantages 55
29.27Disadvantages 55

30 Lecture 09: Overfitting, Bias-Variance Tradeoff, Cross-Validation, and Regularization 56
30.1 Learning Objectives 56
30.2 Introduction 56
30.3 Overfitting 56
30.4 Underfitting 56
30.5 Example Problem 57
30.6 Bias-Variance Tradeoff (Detailed Explanation with Example) 57
30.6.1 Bias (Underfitting) 57
30.6.2 Variance (Overfitting) 57
30.6.3 Bias-Variance Tradeoff 58
30.6.4 Simple Intuition 58
30.6.5 Goal 58
30.7 Cross-Validation 58
30.7.1 Train-Test Split 58
30.7.2 k-Fold Cross-Validation 58
30.8 Regularization (Controlling Overfitting) 58
30.9 L2 Regularization (Ridge Regression) 59
30.10L1 Regularization (Lasso Regression) 59
30.11Key Difference Between L1 and L2 60
30.12Dropout (Neural Networks) 60
30.13Early Stopping 60
30.14Worked Example (Early Stopping) 61
30.15Comparison of Regularization Techniques 61
30.16Model Evaluation and Improvement 61
30.17Confusion Matrix 61
30.18Why Accuracy is Not Enough 62
30.19Precision 62
30.20Recall (Sensitivity) 62
30.21Precision vs Recall Tradeoff 62
30.22F1 Score 63
30.23Worked Example 63
30.24ROC Curve 63
30.25AUC (Area Under Curve) 64
30.26Key Practical Insights 64

31 Lecture 10: K-Means Clustering 65
31.1 Introduction 65
31.2 Real-Life Intuition 65
31.3 Algorithm Steps 65
31.4 Distance Formula (Euclidean) 65

32 Dataset Example 65

33 Step 1: Initialize Centroids **65**

34 Step 2: Distance Calculation Table **66**
34.1 Point A(1,1) . . . 66
34.2 Point B(1.5,2) . . . 66
34.3 Point C(3,4) . . . 66
34.4 Point D(5,7) . . . 66
34.5 Point E(3.5,5) . . . 66
34.6 Point F(4.5,5) . . . 66
34.7 Point G(3.5,4.5) . . . 67

35 Step 3: Cluster Formation **67**

36 Step 4: Update Centroids **67**

37 Step 5: Second Iteration (Distance Recalculation) **67**
37.1 Point A(1,1) . . . 67
37.2 Point B(1.5,2) . . . 68
37.3 Point C(3,4) . . . 68
37.4 Point D(5,7) . . . 68
37.5 Point E(3.5,5) . . . 68
37.6 Point F(4.5,5) . . . 68
37.7 Point G(3.5,4.5) . . . 68

38 Objective Function **68**

39 Advantages **69**

40 Disadvantages **69**

41 Lecture 11: Hierarchical Clustering and Agglomerative Clustering **70**
41.1 Introduction to Clustering . . . 70
41.2 Limitations of K-Means . . . 70
41.3 Hierarchical Clustering . . . 70
41.4 Types of Hierarchical Clustering . . . 70
41.5 Distance Measures . . . 70
41.5.1 Euclidean Distance . . . 70
41.5.2 Manhattan Distance . . . 71
41.5.3 Cosine Similarity . . . 71
41.6 Linkage Methods . . . 71
41.7 Agglomerative Clustering Algorithm . . . 71
41.8 Dendrogram . . . 71
41.9 Worked Example . . . 71
41.9.1 Step 1: Distance Calculations . . . 72
41.9.2 Initial Distance Matrix . . . 72
41.9.3 Step 2: First Merge . . . 72
41.9.4 Step 3: Update Distances (Single Linkage) . . . 72
41.9.5 Updated Matrix . . . 72
41.9.6 Step 4: Second Merge . . . 72
41.9.7 Step 5: Update Distance . . . 72

41.9.8 Final Merge . . . 72
41.10 Dendrogram Representation . . . 73
41.11 Divisive Clustering (Top-Down Approach) . . . 73
41.11.1 Example (Divisive Clustering) . . . 73
41.11.2 Step 1: Compute Pairwise Distances . . . 73
41.11.3 Step 2: Start with One Cluster . . . 73
41.11.4 Step 3: First Split . . . 74
41.11.5 Step 4: Further Splitting of Each Cluster . . . 74
41.11.6 Step 5: Final Splits . . . 74
41.11.7 Final Hierarchy . . . 74
41.12 Applications . . . 74
41.13 Advantages . . . 75
41.14 Disadvantages . . . 75

42 Lecture 12: PCA and Self-Organizing Maps (SOM) 76
42.1 Principal Component Analysis (PCA) . . . 76
42.1.1 Introduction . . . 76
42.1.2 What are Principal Components? . . . 76
42.1.3 Steps of PCA . . . 76
42.1.4 Constructing Covariance Matrix from Dataset . . . 76
42.1.5 Eigenvalue Calculation . . . 78
42.1.6 Interpretation of Eigenvalues . . . 78
42.1.7 Eigenvector (for $\lambda_1 = 4$) . . . 78
42.2 Self-Organizing Maps (SOM) . . . 79
42.2.1 Introduction . . . 79
42.2.2 Key Concepts . . . 79
42.2.3 Distance Measure . . . 80
42.2.4 Weight Update Rule . . . 80
42.2.5 Numerical Problem . . . 80
42.2.6 Initial SOM Structure (Before Training) . . . 80
42.3 Iteration 1 . . . 80
42.3.1 Input A = (1,2) . . . 80
42.3.2 Input B = (2,3) . . . 81
42.3.3 Input C = (5,6) . . . 81
42.4 Iteration 2 . . . 81
42.4.1 Input A = (1,2) . . . 81
42.4.2 Input B = (2,3) . . . 82
42.4.3 Input C = (5,6) . . . 82
42.5 Final Weights After 2 Iterations . . . 82
42.5.1 SOM Neuron Structure (Vertical Representation) . . . 82
42.5.2 Interpretation . . . 83

43 Lecture 13: Reinforcement Learning (RL) 84
43.1 Introduction . . . 84
43.2 Basic RL Interaction . . . 84
43.3 Elements of Reinforcement Learning . . . 84
43.3.1 Agent . . . 84
43.3.2 Environment . . . 84

43.3.3 State . . . 85
43.3.4 Action . . . 85
43.3.5 Reward . . . 85
43.4 Goal of RL . . . 86
43.5 Discount Factor . . . 86
43.6 Application of Deep Learning in RL . . . 86
43.7 Deep Q-Network (DQN) . . . 86
43.8 DQN Architecture . . . 87

44 Markov Decision Process (MDP) 87
44.1 Introduction . . . 87
44.2 Transition Probabilities . . . 87
44.3 Markov Property . . . 88
44.4 Policy . . . 88
44.4.1 Deterministic Policy . . . 88
44.4.2 Stochastic Policy . . . 89
44.5 Value Function . . . 89
44.6 Q-Value Function . . . 89
44.7 Bellman Equation for Value Function . . . 89
44.8 Bellman Equation for Q-Function . . . 90
44.9 Difference Between $V(s)$ and $Q(s, a)$ . . . 90
44.10 Exploration vs Exploitation . . . 90
44.11 ϵ-Greedy Strategy . . . 90
44.12 Q-Learning (Model-Free Reinforcement Learning) . . . 90
44.12.1 Q-Learning Update Rule . . . 91
44.13 Numerical Example: Robot in House (Room Navigation) . . . 91
44.13.1 Scenario . . . 91
44.13.2 Reward Structure . . . 91
44.13.3 Initial Q-Table . . . 91
44.13.4 Episode 1: Learning Step-by-Step . . . 92
44.13.5 Updated Q-Table after Episode 1 . . . 92
44.13.6 Key Insight . . . 93
44.14 Numerical Example: Grid World Navigation (2D Q-Learning) . . . 93
44.14.1 Scenario . . . 93
44.14.2 Grid Description . . . 93
44.14.3 Actions . . . 93
44.14.4 Learning Parameters . . . 93
44.14.5 Episode 1: Step-by-Step Learning . . . 94
44.14.6 Updated Q-Table (After Episode 1) . . . 95
44.15 Relation to DQN . . . 95

1 Lecture 01

Course Details

- Program: BCS
- Course: Machine Learning
- Lecture Duration: 2 Hours
- References:
  - Mitchell, Tom. *Machine Learning*, Chapter 1
  - Alpaydin, Ethem. *Introduction to Machine Learning*, Chapter 1

2 Preliminary Concepts for Machine Learning

Machine Learning relies heavily on concepts from probability, statistics, linear algebra, and optimization. A clear understanding of these foundational ideas is essential for developing intuition about learning algorithms and their behavior. This section reviews the most important preliminary concepts required to study Machine Learning.

2.1 Data, Variables, and Samples

A dataset consists of multiple observations or examples. Each observation is described by a set of variables called features.

Let:

$$x = (x_1, x_2, \ldots, x_n)$$

Each x_i represents a feature.

A single row in a dataset corresponds to one feature vector.

The complete dataset can be represented as a matrix:

$$X = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1n} \\ x_{21} & x_{22} & \ldots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \ldots & x_{mn} \end{bmatrix}$$

In supervised learning, each input vector is associated with an output label:

$$(x, y)$$

Example: In house price prediction, one sample may contain features such as house area, number of rooms, and age of the house, while the label represents the house price.

2.2 Random Variables and Probability

A random variable is a variable whose value depends on the outcome of a random process.

Discrete random variables take countable values.

Continuous random variables take values from an interval.

A probability distribution describes how likely each possible value of a random variable is.

Examples include class labels in classification as discrete random variables and house prices in regression as continuous random variables.

Example: In spam classification, the label "spam" or "not spam" is a discrete random variable, while email length can be treated as a continuous random variable.

2.3 Basic Probability Concepts

2.3.1 Probability of an Event

$$0 \leq P(A) \leq 1$$

$P(A) = 0$ means impossible, and $P(A) = 1$ means certain.

Example: If 30% of emails in a dataset are spam, then the probability that a randomly selected email is spam is 0.3.

2.3.2 Joint Probability

The probability that two events occur together is:

$$P(A, B)$$

Example: The probability that an email is spam and contains the word "free".

2.3.3 Conditional Probability

The probability of event A given that B has occurred is:

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

Conditional probability plays a central role in classification algorithms.

Example: The probability that an email is spam given that it contains the word "offer".

2.4 Bayes' Theorem

Bayes' Theorem provides a way to update prior beliefs based on new evidence:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Where H is a hypothesis, E is observed evidence, $P(H)$ is the prior probability, $P(E|H)$ is the likelihood, and $P(H|E)$ is the posterior probability.

In Machine Learning, the hypothesis corresponds to a class label and evidence corresponds to observed features.

Bayes' Theorem forms the theoretical foundation of Naive Bayes classifiers and probabilistic learning methods.

Example: Naive Bayes spam classifiers compute the probability that an email is spam given the words appearing in it.

2.5 Independence and Conditional Independence

Two events A and B are independent if:

$$P(A, B) = P(A)P(B)$$

Conditional independence is defined as:

$$P(A, B|C) = P(A|C)P(B|C)$$

The Naive Bayes assumption states that features are conditionally independent given the class label.

Example: In Naive Bayes, words in an email are assumed to be independent once the email class is known.

2.6 Statistical Measures

2.6.1 Mean (Arithmetic Mean)

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The mean represents the central tendency of data.

Example: Gaussian Naive Bayes computes the mean of each feature separately for each class.

2.6.2 Geometric Mean

The geometric mean of n positive values is:

$$GM = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}}$$

It is useful when data values grow multiplicatively or when combining probabilities.

Example: The geometric mean is used to evaluate classifier performance on imbalanced datasets.

2.6.3 Variance and Standard Deviation

Variance is defined as:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

Standard deviation is:

$$\sigma = \sqrt{\sigma^2}$$

These measures are important for feature normalization and Gaussian-based models.

Example: Variance controls how widely feature values are spread in Gaussian Naive Bayes models.

2.7 Probability Distributions in Machine Learning

2.7.1 Bernoulli Distribution

Binary outcomes are modeled as:

$$P(X = x) = p^x(1-p)^{1-x}$$

Used in binary classification and logistic regression.

Example: Logistic regression predicts the probability of binary outcomes such as pass or fail.

2.7.2 Gaussian Distribution

$$P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Used in Naive Bayes classifiers and noise modeling.

Example: Continuous features like height or sensor values are often modeled using Gaussian distributions.

2.8 Linear Algebra Essentials

2.8.1 Vectors

A feature vector is written as:

$$x = (x_1, x_2, \ldots, x_n)$$

Example: Each row of a dataset is treated as a vector during training.

2.8.2 Dot Product

$$w \cdot x = \sum_{i=1}^{n} w_i x_i$$

This operation appears in linear regression, logistic regression, and neural networks.

Example: Linear regression computes predictions using the dot product between weights and inputs.

2.8.3 Matrix Representation

Datasets are represented as matrices where rows correspond to samples and columns correspond to features.

Example: Neural networks rely heavily on matrix multiplication for efficient computation.

2.9 Functions and Models

A Machine Learning model is defined as:

$$y = f(x; \theta)$$

Learning is the process of finding parameters θ that best map inputs to outputs.

Example: Linear regression models data using a linear function with learned parameters.

2.10 Loss Functions and Error Measurement

A loss function measures how wrong a prediction is.

Mean Squared Error is defined as:

$$L = \frac{1}{n} \sum (y - \hat{y})^2$$

Loss functions define what constitutes good performance.

Example: Regression models are trained by minimizing the mean squared error.

2.11 Optimization and Learning

Learning involves minimizing loss:

$$\theta^* = \arg\min_{\theta} L(\theta)$$

This leads to optimization techniques such as Gradient Descent and Stochastic Gradient Descent.

Example: Neural networks update parameters using gradient descent.

2.12 Generalization and Overfitting

Training error measures performance on seen data, while test error measures performance on unseen data.

Overfitting occurs when a model learns noise instead of patterns.

Generalization refers to a model's ability to perform well on new data.

Example: A model with high training accuracy but low test accuracy is overfitting.

3 Introduction to Machine Learning

3.1 Definition

Machine Learning (ML) is the study of computer algorithms that improve automatically through experience. According to Tom Mitchell:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." [**?**]

Instructor Notes: Explain slowly, break into 3 parts:

- T = task (what the program is trying to do)
- P = performance measure (how we know it is successful)
- E = experience (data, interactions, feedback)

3.2 Examples

- Email spam filtering: Program improves by learning from emails labeled as spam/not spam
- Predicting electricity load: Program predicts next day demand based on historical load
- Handwriting recognition: Program learns from labeled images of letters
- Recommendation system: Program learns user preferences to suggest movies/products

3.3 Applications of Machine Learning

- **Finance:** Fraud detection, credit scoring
- **Healthcare:** Disease prediction, medical image analysis
- **Marketing:** Customer segmentation, churn prediction
- **Energy:** Load forecasting, anomaly detection in power grids

4 Comparison: AI, Machine Learning, and Deep Learning

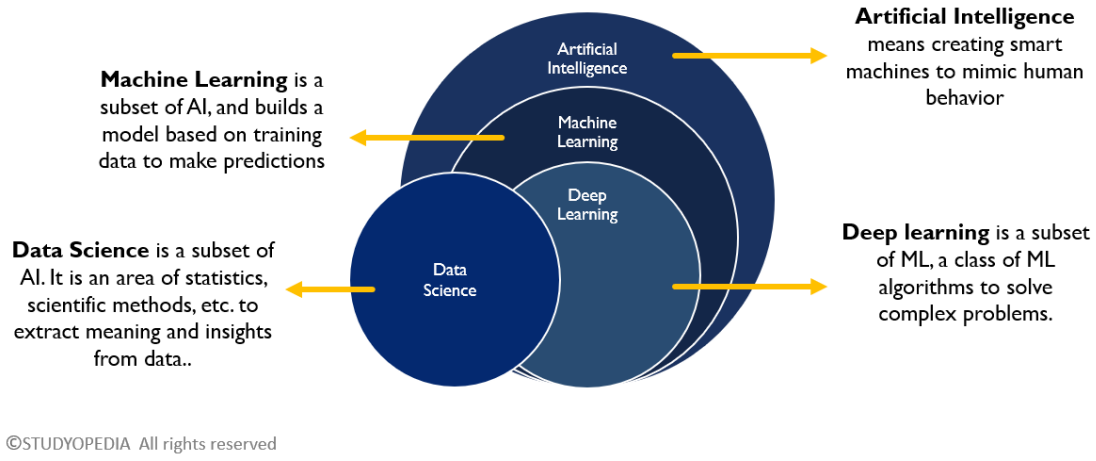


Figure 1: Relationship between AI, ML, and Data Science (placeholder)

- AI = broad concept of machines performing intelligent tasks
- ML = subset of AI where machines improve automatically from data
- Data Science = broader field encompassing ML, data analysis, statistics, visualization

4.1 Scenarios

- AI: Self-driving car
- ML: Car predicts pedestrian movement using past data
- Data Science: Analyze traffic patterns from large datasets

5 Learning Paradigms

5.1 1. Supervised Learning

Definition: Learning from labeled data to predict output. **Example:** Predict house prices from historical data **Data Example:**

House Size (sqft)	Bedrooms	Age (years)	Price ($1000)
2104	3	45	460
1600	3	40	330
2400	3	20	540
1416	2	30	232
3000	4	10	610

Show how features (size, bedrooms, age) are mapped to output (price).

5.2 2. Unsupervised Learning

Definition: Learning patterns from unlabeled data **Example:** Customer segmentation **Scenario:** Group customers by buying habits without knowing segments

5.3 3. Semi-Supervised Learning

Definition: Small labeled dataset + large unlabeled dataset **Example:** Image classification when labeling all images is expensive

5.4 4. Reinforcement Learning

Definition: Learning via rewards/punishments **Example:** Robot navigation, AlphaGo **Scenario:** Agent learns optimal path in a maze based on feedback

6 Case Studies

6.1 1. Playing Checkers

- Task T: Play checkers
- Performance measure P: Fraction of games won
- Experience E: Past games played by program

6.2 2. Speech Recognition

- Task T: Recognize spoken words
- Performance measure P: Accuracy of recognition
- Experience E: Recorded audio with transcripts

6.3 3. Handwriting Recognition

- Task T: Identify handwritten characters
- Performance measure P: Fraction of correct predictions
- Experience E: Dataset of labeled images (20x20 pixels)

7 Questions

Short Questions

1. What is Machine Learning? **Solution:** Machine Learning is the study of algorithms that improve their performance on a task through experience (data) without being explicitly programmed.
2. What is a feature in a dataset? **Solution:** A feature is an individual measurable property or attribute of a data sample, represented as a variable.
3. Define a label. **Solution:** A label is the output or target value associated with an input sample in supervised learning.
4. What is overfitting? **Solution:** Overfitting occurs when a model learns noise or details from training data and performs poorly on unseen data.
5. State Bayes' Theorem. **Solution:**

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

6. What does conditional independence mean in Naive Bayes? **Solution:** Features are assumed to be independent of each other given the class label.

Conceptual Exercises

1. Explain Tom Mitchell's definition of Machine Learning using:
   - Task (T)
   - Experience (E)
   - Performance Measure (P)

   Solution: T is the objective (e.g., classify emails), E is the dataset or past examples, P is a metric such as accuracy or error rate used to evaluate improvement.

2. Differentiate between supervised and unsupervised learning. **Solution:** Supervised learning uses labeled data and learns input-output mappings, while unsupervised learning uses unlabeled data to discover patterns or structure.

3. Why is generalization important in Machine Learning? **Solution:** Generalization ensures that a model performs well on unseen data rather than memorizing training examples.

4. Give one example where reinforcement learning is more suitable than supervised learning. **Solution:** Robot navigation, where feedback is received as rewards rather than labeled examples.

Numerical and Applied Problems

1. A dataset contains 500 samples, out of which 150 belong to Class A.

   (a) Compute $P(\text{Class A})$.

   Solution:
   $$P(\text{Class A}) = \frac{150}{500} = 0.3$$

2. Given data points:
   $$x = \{2, 4, 6, 8\}$$

   (a) Compute the mean.

   (b) Compute the variance.

   Solution:
   $$\mu = \frac{2+4+6+8}{4} = 5$$
   $$\sigma^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5$$

3. In a spam dataset, $P(\text{spam}) = 0.4$ and $P(\text{free}|\text{spam}) = 0.5$. Interpret $P(\text{free}|\text{spam})$.

   Solution: Given that an email is spam, there is a 50% chance that it contains the word "free".

4. A model achieves 95% accuracy on training data and 60% on test data.

   (a) Identify the issue.

   (b) Suggest one remedy.

   Solution: The model is overfitting. Remedies include using more data, regularization, or reducing model complexity.

5. Write a linear regression model for house price prediction with features size (x_1) and age (x_2).

   Solution:
   $$y = w_1 x_1 + w_2 x_2 + b$$

8 Lecture 02: Types of Learning

Machine Learning systems differ not only in algorithms but also in how they acquire knowledge, update models, and represent learned information. Understanding these learning paradigms is fundamental before studying specific algorithms.

8.1 Inductive Learning

Inductive learning is the process of inferring a general hypothesis from specific observed examples.

Given training data:

$$D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

the learner searches the hypothesis space H to find a function $h \in H$ such that:

$$h(x_i) \approx y_i \quad \forall i = 1, \ldots, n$$

Induction moves from:

$$\text{Specific Observations} \rightarrow \text{General Hypothesis}$$

Inductive Learning Hypothesis

A fundamental assumption in Machine Learning:

If a hypothesis approximates the target function well over a sufficiently large training set, it will also approximate it well over unseen examples drawn from the same distribution.

This assumption is probabilistic, not logically guaranteed.

Technical Insight: Inductive learning assumes that the training and test data are drawn from the same underlying probability distribution $P(X, Y)$.

Example

In fraud detection: The system observes thousands of labeled transactions (fraud / not fraud). It infers a general decision function. There is no logical proof that the next transaction follows the same pattern — we assume statistical regularity.

Inductive Bias

Inductive bias is the set of assumptions that restrict or guide the hypothesis search.

Without bias, learning is impossible because infinitely many hypotheses can perfectly fit finite training data.

Types of bias:

- **Restriction Bias:** Limits the hypothesis space. Example: Linear regression restricts solutions to linear functions.
- **Preference Bias:** Prefers certain hypotheses over others. Example: Decision trees prefer shorter trees (Occam's Razor).

Technical Example: Naive Bayes assumes conditional independence given the class label y:

$$P(x_1, x_2, \ldots, x_n \mid y) = \prod_{i=1}^{n} P(x_i \mid y)$$

This assumption simplifies computation dramatically but introduces bias.

8.2 Deductive Learning

Deductive learning derives specific conclusions from general principles.

It moves from:

$$\text{General Rule} \rightarrow \text{Specific Conclusion}$$

If premises are true, conclusions must be true.

Technical Difference:

$$\text{Induction: } Data \rightarrow Hypothesis$$

$$\text{Deduction: } Rules + Facts \rightarrow LogicalConclusion$$

Example:

In rule-based expert systems (medical diagnosis before ML era): If rules state: "IF fever AND rash THEN measles," the system deduces the diagnosis logically.

Machine Learning does not start with such rules — it learns them from data.

8.3 Online (Incremental) Learning

Online learning updates model parameters after each new training example.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; x_t, y_t)$$

where:

- η is the learning rate
- ∇L is the gradient computed using the current example

Key Property: The model evolves continuously.

Example: In recommendation systems (e.g., streaming platforms): When a user clicks or skips content, the model updates immediately to refine personalization.

Technical Note: Online learning is essential when data distribution changes over time (concept drift).

8.4 Batch (Offline) Learning

Batch learning trains the model using the entire dataset simultaneously.

$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{n} L(h(x_i), y_i)$$

where L is an appropriate loss function (e.g., MSE for regression, cross-entropy for classification).

Example: In credit scoring: Banks train models using historical customer data once per quarter, validate performance, and deploy the fixed model.

Technical Insight: Batch learning generally provides more stable convergence because gradient estimates use the full dataset.

8.5 Instance-Based Learning

Instance-based learning stores training examples and delays generalization until prediction time (also called **lazy learning**).

Prediction for new input x depends directly on similarity to stored examples.

Example distance metric (Euclidean):

$$d(x_i, x) = \sqrt{\sum_{j=1}^{d} (x_{ij} - x_j)^2}$$

Example: In document retrieval systems: New query documents are compared with stored documents using similarity metrics (e.g., cosine similarity) to retrieve the most relevant ones.

Technical Property: Decision boundaries are highly flexible and data-dependent.

- Memory complexity: $O(n \cdot d)$
- Prediction complexity: $O(n \cdot d)$

8.6 Model-Based Learning

Model-based learning constructs an explicit parametric or non-parametric model from training data (also called **eager learning**).

Example (Linear Model):

$$y = w^T x + b$$

Model parameters are learned by minimizing a loss function.

Example: In house price prediction: After training, only parameters w and b are stored; raw training data is no longer needed for prediction.

Technical Insight: Training may be computationally expensive, but prediction is efficient:

$$\text{Prediction complexity} = O(d)$$

where d is the number of features.

8.7 Instance-Based vs Model-Based (Technical View)

Aspect	Instance-Based	Model-Based
Learning Strategy	Lazy	Eager
Training Cost	Low	High
Prediction Cost	High ($O(n \cdot d)$)	Low ($O(d)$)
Memory Requirement	Stores data ($O(n \cdot d)$)	Stores parameters
Generalization	Local	Global

8.8 Supervised Learning Basics: Classification vs Regression

Reference: Mitchell, Chapter 3

Supervised learning is the most common learning paradigm in Machine Learning. In supervised learning, each training example is labeled with a target value y. The goal is to learn a function $h : X \to Y$ that predicts y for unseen inputs x.

Classification

Classification predicts discrete labels or categories.

$$h : X \to \{C_1, C_2, \ldots, C_k\}$$

where $C_1, \ldots, C_k$ are possible classes.

Example:

- Email spam detection: $y \in$ {spam, not spam}
- Fraud detection: $y \in$ {fraud, not fraud}
- Image recognition: $y \in$ {cat, dog, bird, ...}

Loss Function for Classification: Commonly cross-entropy loss:

$$L(h(x), y) = -\sum_{i=1}^{k} y_i \log p_i$$

where p_i is the predicted probability for class i.

Regression

Regression predicts continuous values.

$$h : X \to \mathbb{R}$$

Example:

- House price prediction: $y =$ price in dollars
- Temperature forecasting: $y =$ degrees Celsius
- Stock market prediction: $y =$ future price

Loss Function for Regression: Commonly mean squared error (MSE):

$$L(h(x), y) = \frac{1}{n} \sum_{i=1}^{n} (h(x_i) - y_i)^2$$

Key Differences Between Classification and Regression

Aspect	Classification	Regression
Output Type	Discrete labels	Continuous values
Typical Loss	Cross-entropy, 0-1 loss	MSE, MAE
Goal	Assign class labels	Predict numerical quantity
Examples	Spam detection, Image recognition	House prices, Temperature

Technical Note: Some algorithms, such as decision trees or neural networks, can handle both regression and classification tasks by adjusting the output layer and loss function.

8.9 Popular Machine Learning Algorithms: Supervised vs Unsupervised

Supervised Learning	**Unsupervised Learning**
Linear Regression	K-Means Clustering
Logistic Regression	Hierarchical Clustering
Decision Trees (CART, ID3, C4.5)	DBSCAN
Random Forests	Gaussian Mixture Models (GMM)
Support Vector Machines (SVM)	Principal Component Analysis (PCA)
k-Nearest Neighbors (k-NN)	t-SNE (Dimensionality Reduction)
Gradient Boosting (XGBoost, LightGBM)	Autoencoders (unsupervised feature learning)
Naive Bayes	Association Rule Mining (Apriori, FP-Growth)
Neural Networks (Supervised)	Self-Organizing Maps (SOM)

Technical Note: Some algorithms, such as neural networks or clustering methods, can be adapted for both supervised and unsupervised tasks depending on the objective and available labels.

8.10 K-Nearest Neighbors (k-NN): Distance Metrics and Example Problems

Reference: Mitchell, Chapter 3

The **k-Nearest Neighbors (k-NN)** algorithm is a simple, yet powerful supervised learning method. It can be used for both classification and regression tasks.

Basic Idea

Given a new input x, k-NN predicts its output based on the labels of the k closest training examples.

$$\hat{y}(x) = \begin{cases} \text{majority vote of } k \text{ neighbors} & \text{for classification} \\ \text{average of } k \text{ neighbors' values} & \text{for regression} \end{cases}$$

Key Property: k-NN is a **lazy learner**, meaning it does not build an explicit model during training. It stores all data and performs computation at prediction time.

Distance Metrics

The similarity between data points is measured using a distance metric. Common choices:

1. **Euclidean Distance**

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^{d} (x_{il} - x_{jl})^2}$$

2. **Manhattan Distance**

$$d(x_i, x_j) = \sum_{l=1}^{d} |x_{il} - x_{jl}|$$

3. **Minkowski Distance**

$$d(x_i, x_j) = \left(\sum_{l=1}^{d} |x_{il} - x_{jl}|^p \right)^{1/p}$$

Special cases: $p = 1 \rightarrow$ Manhattan, $p = 2 \rightarrow$ Euclidean

4. **Cosine Similarity**

$$\text{sim}(x_i, x_j) = \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|}$$

Distance can be defined as $1 - \text{sim}(x_i, x_j)$

Choosing k

- **Small k:** Sensitive to noise, may overfit - **Large k:** Smoother predictions, may underfit - Rule of thumb: $k \approx \sqrt{n}$, where n is number of training examples

Example Problem: Classification

Problem: We have a dataset of points labeled as 'Red' or 'Blue'. Features are 2D coordinates (x_1, x_2):

Point	Coordinates (x_1, x_2)	Label
A	(1,2)	Red
B	(2,3)	Red
C	(3,3)	Blue
D	(5,1)	Blue

We want to classify a new point $X = (3, 2)$ using $k = 3$ and Euclidean distance.
Step 1: Compute distances

$$d(X, A) = \sqrt{(3-1)^2 + (2-2)^2} = \sqrt{4+0} = 2$$
$$d(X, B) = \sqrt{(3-2)^2 + (2-3)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$
$$d(X, C) = \sqrt{(3-3)^2 + (2-3)^2} = \sqrt{0+1} = 1$$
$$d(X, D) = \sqrt{(3-5)^2 + (2-1)^2} = \sqrt{4+1} = \sqrt{5} \approx 2.24$$

Step 2: Identify k nearest neighbors
- $k = 3$ closest points: C (Blue, 1), B (Red, 1.41), A (Red, 2)
Step 3: Majority vote
- Labels of neighbors: Blue, Red, Red $\rightarrow$ Majority = **Red**
Result: The new point $X = (3, 2)$ is classified as **Red**.

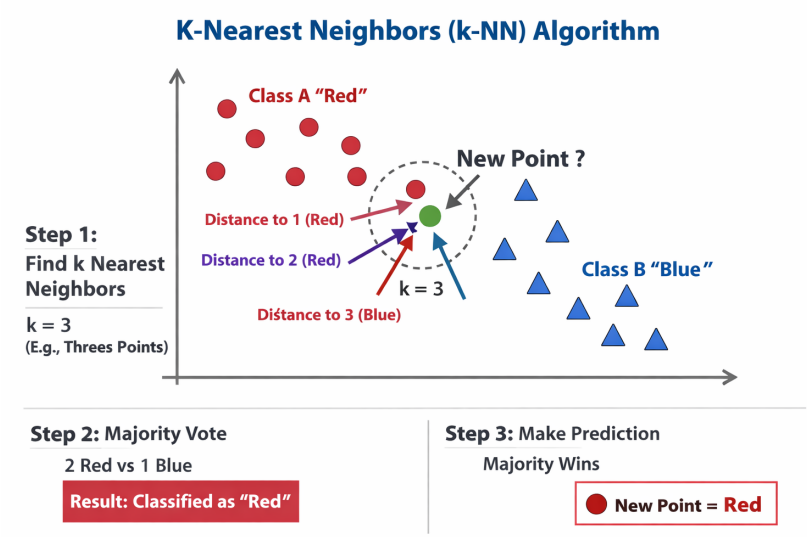


Figure 2: Illustration of the K-Nearest Neighbors (k-NN) algorithm. The new point is classified based on majority vote among its k nearest neighbors.

Example Problem: Regression

Problem: Predict the price of a house based on 1D feature 'Size (1000 sq.ft)' using $k = 2$ nearest neighbors:

House Size	Price ($1000)
1200	200
1500	250
1800	300

Predict price for new house of size 1600 using Euclidean distance:
Step 1: Compute distances

$$d(1600, 1200) = |1600 - 1200| = 400$$
$$d(1600, 1500) = |1600 - 1500| = 100$$
$$d(1600, 1800) = |1600 - 1800| = 200$$

Step 2: Identify k nearest neighbors ($k = 2$)
- Neighbors: 1500 (250k), 1800 (300k)
Step 3: Average their prices

$$\hat{y} = \frac{250 + 300}{2} = 275$$

Result: Predicted house price = $275,000

Summary: k-NN is simple, interpretable, and effective for small to medium datasets. Its performance depends heavily on the choice of distance metric, k, and feature preprocessing.

9 Lecture 03: Decision Trees

Decision Trees are a supervised learning method used for classification and regression tasks. They are easy to interpret and visualize and work by recursively splitting the dataset into subsets based on feature values.

9.1 Entropy

Entropy measures the impurity or uncertainty in a dataset. For a dataset S with c classes:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

Where:

- p_i = proportion of samples in class i in S
- $H(S) = 0$ when all samples belong to a single class (pure)
- $H(S) = 1$ when classes are equally mixed (maximum impurity)

9.2 Information Gain (IG)

Information Gain measures the reduction in entropy after splitting a dataset S on feature A:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- S_v = subset of S where feature A has value v
- $|S_v|$ = number of samples in subset S_v
- IG quantifies how much uncertainty is reduced by splitting on A
- Feature with **highest IG** is chosen for the split

9.3 Gini Index

Gini Index is another measure of impurity of a dataset:

$$Gini(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Where:

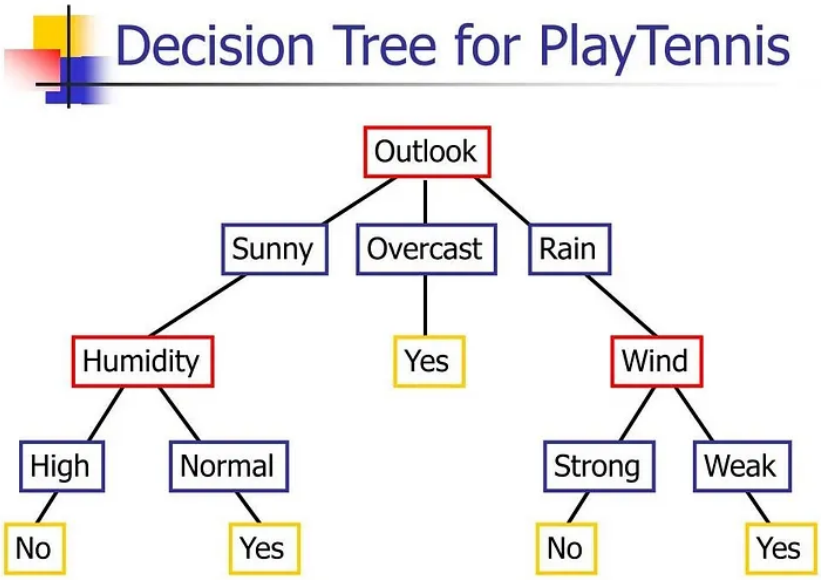
- p_i = proportion of class i in S
- $Gini(S) = 0$ when all samples belong to a single class
- Lower Gini means a **purer split**
- Decision Trees can also use Gini Index to select the best feature

9.4 Dataset: Play Tennis

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

9.5 Tree Structure

- **Internal Nodes:** Represent decisions based on features.
- **Branches:** Represent outcomes of those decisions.
- **Leaf Nodes:** Represent class labels (classification) or values (regression).



9.6 Step 1: Entropy of Entire Dataset

Notation:

$$S = \text{entire dataset}, \quad p_{Yes} = \frac{\#\text{ Yes}}{|S|}, \quad p_{No} = \frac{\#\text{ No}}{|S|}$$

$$H(S) = -p_{Yes} \log_2 p_{Yes} - p_{No} \log_2 p_{No}$$

Number of samples: 14, Yes = 9, No = 5

$$p_{Yes} = \frac{9}{14}, \quad p_{No} = \frac{5}{14}$$

$$H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx -0.643 \cdot \log_2 0.643 - 0.357 \cdot \log_2 0.357 \approx 0.940$$

9.7 Step 2: Entropy for Each Feature

9.7.1 Outlook

- Sunny: 2 Yes, 3 No $\rightarrow |S_{Sunny}| = 5$

$$H(S_{Sunny}) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \approx 0.971$$

- Overcast: 4 Yes, 0 No $\rightarrow |S_{Overcast}| = 4$

$$H(S_{Overcast}) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0$$

- Rain: 3 Yes, 2 No $\rightarrow |S_{Rain}| = 5$

$$H(S_{Rain}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \approx 0.971$$

9.7.2 Information Gain for Outlook

$$IG(S, Outlook) = H(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} H(S_v)$$

$$IG(S, Outlook) = 0.940 - \left(\frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 \right)$$

$$IG(S, Outlook) = 0.940 - (0.347 + 0 + 0.347) = 0.940 - 0.694 = 0.246$$

9.7.3 Humidity

- High: 3 Yes, 4 No $\rightarrow |S_{High}| = 7$

$$H(S_{High}) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} \approx 0.985$$

- Normal: 6 Yes, 1 No $\rightarrow |S_{Normal}| = 7$

$$H(S_{Normal}) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} \approx 0.592$$

$$IG(S, Humidity) = H(S) - \left(\frac{7}{14} \cdot 0.985 + \frac{7}{14} \cdot 0.592 \right) = 0.940 - (0.492 + 0.296) = 0.940 - 0.788 = 0.152$$

9.7.4 Wind

- Weak: 6 Yes, 2 No $\rightarrow |S_{Weak}| = 8$

$$H(S_{Weak}) = -\frac{6}{8}\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} \approx 0.811$$

- Strong: 3 Yes, 3 No $\rightarrow |S_{Strong}| = 6$

$$H(S_{Strong}) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = 1$$

$$IG(S, Wind) = H(S) - \left(\frac{8}{14}\cdot 0.811 + \frac{6}{14}\cdot 1\right) = 0.940 - (0.463 + 0.429) = 0.940 - 0.892 = 0.048$$

9.8 Step 3: Choose Best Feature for Root

Highest Information Gain: $IG(S, Outlook) = 0.246 \rightarrow$ Split on **Outlook**

9.9 Step 4: Split Dataset and Repeat Recursively

- **Overcast:** All Yes $\rightarrow$ leaf node
- **Sunny:** 2 Yes, 3 No $\rightarrow$ split on Humidity (IG higher than Wind)
- **Rain:** 3 Yes, 2 No $\rightarrow$ split on Wind

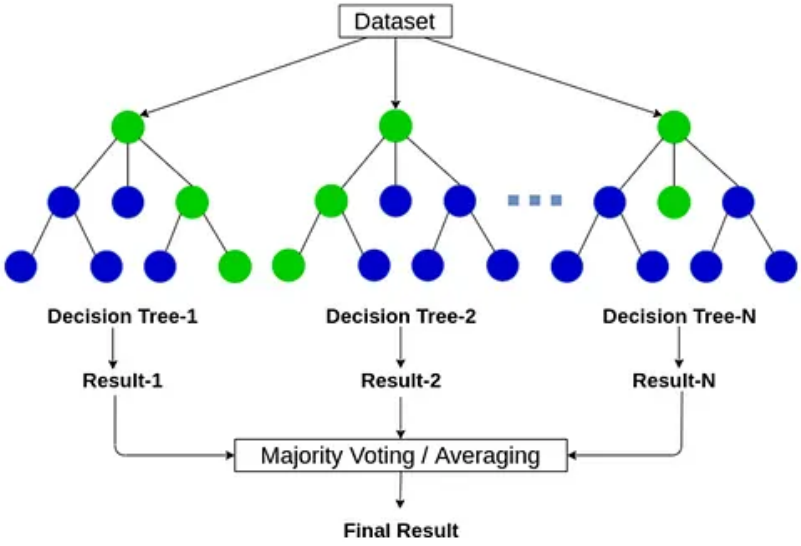
10 Random Forests

Random Forest is an ensemble learning method that builds multiple decision trees and merges their results to improve accuracy and control overfitting. It is mainly used for classification and regression tasks.

10.1 Key Concepts

- **Ensemble Method:** Combines predictions from multiple models.
- **Bagging (Bootstrap Aggregation):** Random subsets of the dataset are used to train each tree.
- **Random Feature Selection:** At each split, a random subset of features is considered, increasing diversity among trees.
- **Voting / Averaging:** Final prediction is based on majority vote (classification) or average (regression) of all trees.

Random Forest



10.2 Algorithm Steps

1. For $b = 1$ to B (number of trees):
   (a) Draw a bootstrap sample S_b of size N from the training data.
   (b) Grow a decision tree T_b on S_b:
   - At each node, select m features at random from the total M features.
   - Split the node using the feature that gives the best split (Entropy / Gini).
2. Output the ensemble of trees $\{T_1, T_2, ..., T_B\}$.
3. Make predictions:

$$\hat{y} = \text{majority vote of } T_1(x), T_2(x), ..., T_B(x) \quad \text{(classification)}$$

$$\hat{y} = \frac{1}{B} \sum_{b=1}^{B} T_b(x) \quad \text{(regression)}$$

10.3 Advantages of Random Forest

- Reduces overfitting compared to a single decision tree.
- Handles high-dimensional data well.
- Can estimate feature importance.
- Robust to noisy data and missing values.

10.4 Disadvantages

- Less interpretable than a single decision tree.
- Larger memory and computation requirements.
- Can be slower for real-time predictions.

10.5 Numerical Example: Random Forest on Play Tennis Dataset

Consider the same Play Tennis dataset with 14 samples. We build a Random Forest with 3 trees (for simplicity). Each tree is trained on a **bootstrap sample** (random subset with replacement), and splits are selected using **Information Gain**.

10.5.1 Step 1: Generate Bootstrap Samples

Tree	Sampled Days	Notes
Tree 1	1, 2, 3, 4, 5, 6, 7, 2, 3, 8, 9, 10, 11, 12	some duplicates due to sampling with replacement
Tree 2	2, 4, 5, 6, 7, 8, 9, 10, 1, 3, 12, 13, 14, 2	duplicates
Tree 3	3, 5, 7, 9, 11, 13, 1, 2, 4, 6, 8, 10, 12, 14	all days selected once

10.5.2 Step 2: Build Decision Trees on Each Sample

Tree 1: Best root = Outlook → Split: Sunny / Overcast / Rain
Tree 2: Best root = Humidity → Split: High / Normal
Tree 3: Best root = Outlook → Split: Sunny / Overcast / Rain
Each tree continues splitting until leaves are pure.

10.6 Tree Splits and Calculations

Tree 1 (Root: Outlook)

Outlook	Yes	No	Total	Entropy
Sunny	2	3	5	0.971
Overcast	4	0	4	0
Rain	3	2	5	0.971

$$IG(S, Outlook) = 0.940 - \left(\frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971\right) = 0.246$$

Sub-splits: Sunny → Humidity (High / Normal) Rain → Wind (Weak / Strong) Overcast → Leaf node (Yes)

Tree 2 (Root: Humidity)

Humidity	Yes	No	Total	Entropy
High	3	4	7	0.985
Normal	6	1	7	0.592

$$IG(S, Humidity) = 0.940 - \left(\frac{7}{14} \cdot 0.985 + \frac{7}{14} \cdot 0.592\right) = 0.152$$

Sub-splits: High → Wind Normal → Leaf node (Yes)

Tree 3 (Root: Outlook)

Similar to Tree 1 (different sampled data).

10.6.1 Step 3: Make Predictions for a New Day

Suppose we want to predict **PlayTennis** for a new day:

- Outlook = Sunny
- Temperature = Hot
- Humidity = High
- Wind = Weak

Predictions by each tree:

Tree	Prediction
Tree 1	No
Tree 2	No
Tree 3	Yes

10.6.2 Step 4: Final Prediction by Majority Vote

$$\hat{y} = \text{majority vote of predictions } = No$$

10.6.3 Step 5: Advantages Observed

- Even if a single tree misclassifies, ensemble voting gives the correct result more often.
- Randomness in both data and feature selection reduces overfitting.

Note: In a real Random Forest, we use many trees (e.g., 100) for more stable predictions.

11 Lecture 05

12 Linear Regression and Logistic Regression

Machine learning algorithms can be categorized into:

- **Regression**: Predict continuous numerical values.
- **Classification**: Predict discrete classes.

Two fundamental supervised learning models are:

- Linear Regression
- Logistic Regression

Linear regression predicts continuous outputs, while logistic regression predicts probabilities for classification tasks.

12.1 Linear Regression

Linear regression models the relationship between input and output using a straight line:

$$y = w_0 + w_1 x$$

Where:

- y = predicted value
- x = input feature
- w_1 = slope (weight)
- w_0 = intercept

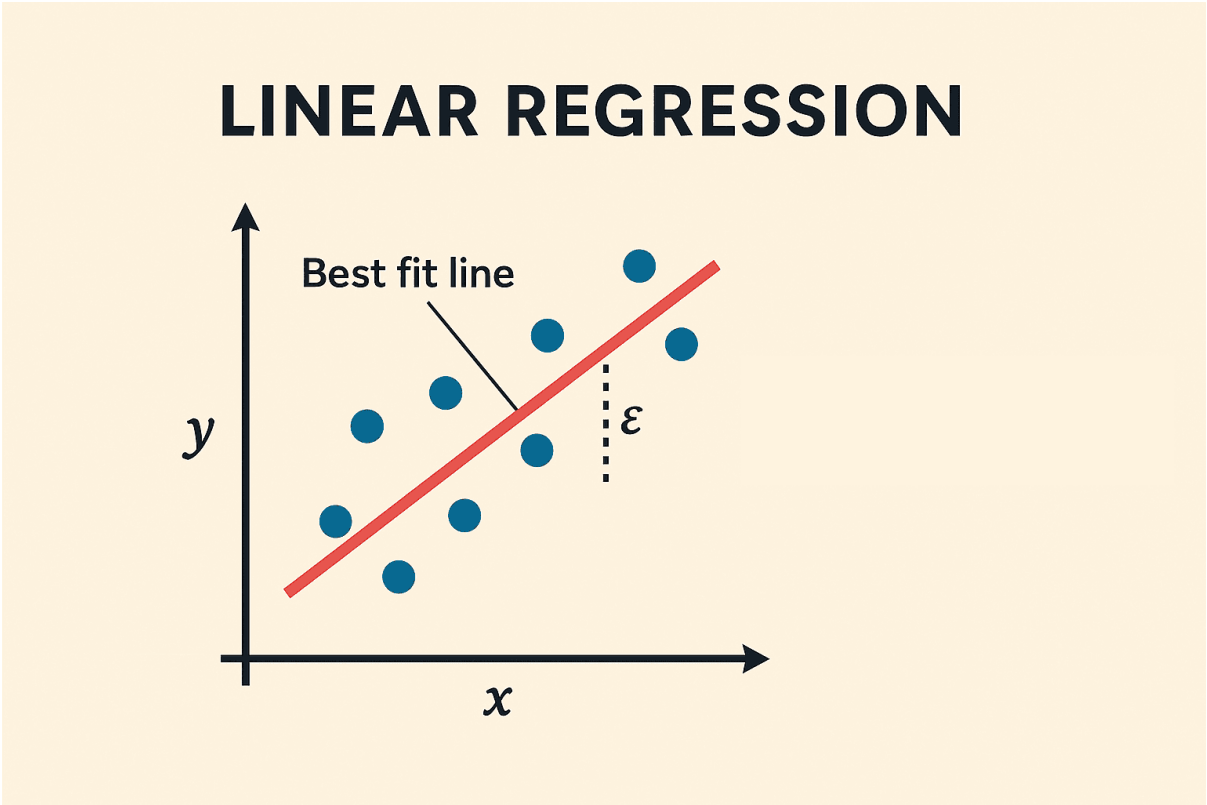
For multiple features:

$$y = w_0 + w_1 x_1 + w_2 x_2 + ... + w_n x_n$$

The goal is to find the parameters that best fit the training data.

12.2 Example Dataset

x	y
1	2
2	3
3	5
4	4



12.3 Step 1: Compute Means

$$\bar{x} = \frac{1+2+3+4}{4} = 2.5$$

$$\bar{y} = \frac{2+3+5+4}{4} = 3.5$$

12.4 Step 2: Compute Slope

Slope formula:

$$w_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x - \bar{x})(y - \bar{y})$	$(x - \bar{x})^2$
1	2	-1.5	-1.5	2.25	2.25
2	3	-0.5	-0.5	0.25	0.25
3	5	0.5	1.5	0.75	0.25
4	4	1.5	0.5	0.75	2.25
Sum				**4**	**5**

$$w_1 = \frac{4}{5} = 0.8$$

12.5 Step 3: Compute Intercept

$$w_0 = \bar{y} - w_1\bar{x}$$

$$w_0 = 3.5 - (0.8)(2.5)$$

$$w_0 = 3.5 - 2$$

$$w_0 = 1.5$$

12.6 Final Linear Regression Model

$$y = 1.5 + 0.8x$$

12.7 Prediction Example

For $x = 5$

$$y = 1.5 + 0.8(5)$$

$$y = 5.5$$

Predicted value = **5.5**

12.8 Logistic Regression

Linear regression cannot be used for classification because outputs can be any real value. Logistic regression solves this by converting the output to a probability.

First compute:

$$z = w_0 + w_1 x$$

Then apply the sigmoid function.

12.9 Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

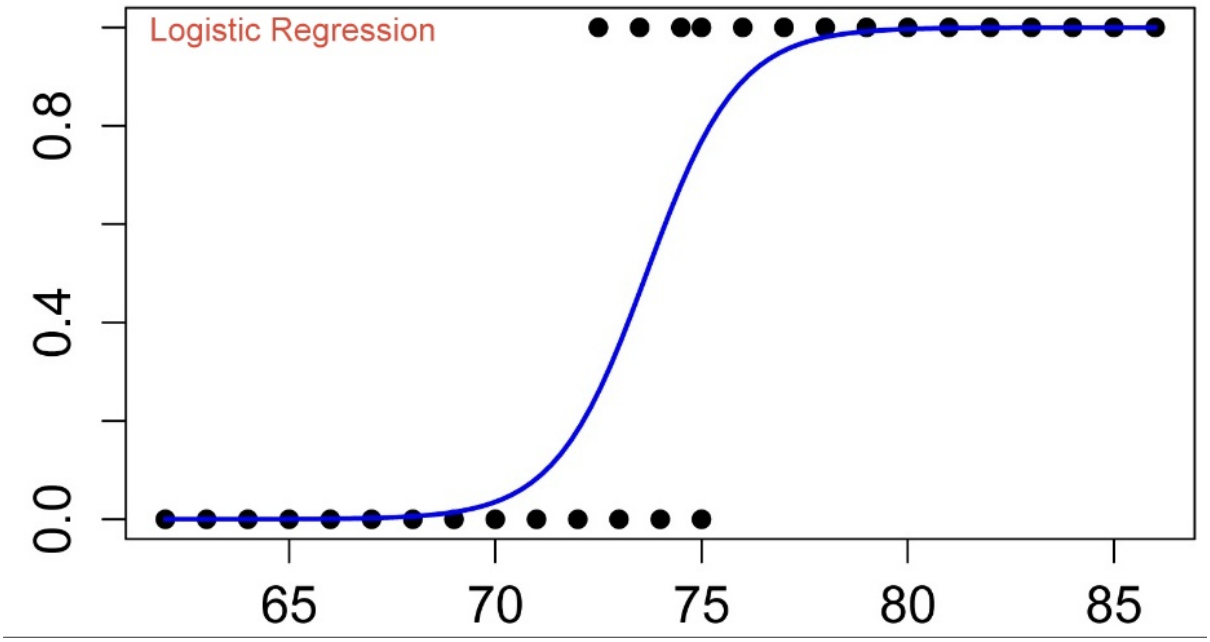
This maps any value into the range:

$$0 \leq P \leq 1$$

The decision rule is:

$$P(y = 1|x) > 0.5 \Rightarrow \text{Class } 1$$

$$P(y = 1|x) \leq 0.5 \Rightarrow \text{Class } 0$$



12.10 Numerical Example

Model:

$$z = -4 + 1.2x$$

For $x = 5$

$$z = -4 + 1.2(5)$$

$$z = 2$$

Apply sigmoid:

$$P = \frac{1}{1 + e^{-2}}$$

$$e^{-2} \approx 0.135$$

$$P = \frac{1}{1.135}$$

$$P \approx 0.88$$

Probability of passing = **88%**
Since $P > 0.5$, prediction = **Pass**.

12.11 Learning Model Parameters using Gradient Descent

In machine learning models such as linear regression and logistic regression, the parameters (weights) are not chosen manually. Instead, they are learned automatically from the training data using an optimization algorithm called **Gradient Descent**.

The objective of training is to minimize a **loss function**, which measures the difference between the predicted output and the true output.

12.12 Loss Function

For logistic regression, the loss function is the **Log Loss (Cross-Entropy Loss)**:

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

where:

- y = true label (0 or 1)
- $p = \sigma(z) = \frac{1}{1+e^{-z}}$ = predicted probability
- $z = w_0 + w_1 x_1 + \cdots + w_n x_n$

The gradient with respect to a weight w_j is:

$$\frac{\partial L}{\partial w_j} = (p - y) x_j$$

and the weight update rule is:

$$w_j := w_j - \alpha \frac{\partial L}{\partial w_j} = w_j - \alpha (p - y) x_j$$

where α is the learning rate.

12.12.1 Training Procedure

The training process using gradient descent follows these steps:

1. Initialize the model weights randomly.
2. Compute the predicted output using the model.
3. Calculate the loss using the loss function.
4. Compute the gradient of the loss with respect to the weights.
5. Update the weights using the gradient descent update rule.
6. Repeat the process until the loss converges to a minimum.

12.13 Numerical Example

Dataset (single data point):

Feature x_1	Label y
2	1

Initial Weights and Learning Rate:

$$w_0 = 0.1, \quad w_1 = 0.2, \quad \alpha = 0.1$$

Step 1: Linear combination

$$z = w_0 + w_1 x_1 = 0.1 + 0.2 \cdot 2 = 0.5$$

Step 2: Apply sigmoid

$$p = \sigma(z) = \frac{1}{1 + e^{-0.5}} \approx 0.622$$

Step 3: Compute error

$$\text{Error} = p - y = 0.622 - 1 = -0.378$$

Step 4: Update weights

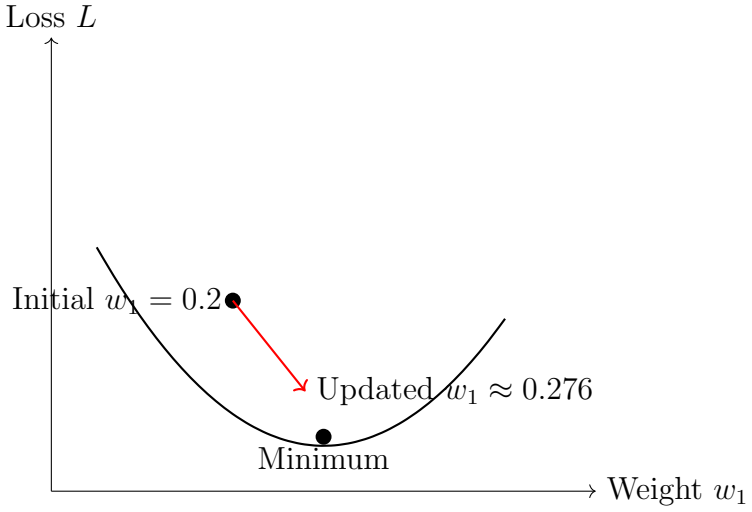
$$w_0^{\text{new}} = w_0 - \alpha(p - y) = 0.1 - 0.1(-0.378) \approx 0.138$$

$$w_1^{\text{new}} = w_1 - \alpha(p - y)x_1 = 0.2 - 0.1(-0.378 \cdot 2) \approx 0.276$$

Step 5: Summary of weight update

Weight	Old	Updated
w_0	0.100	0.138
w_1	0.200	0.276

12.14 Visualization of Gradient Step



Explanation: The weight w_1 moves from its initial value towards the minimum of the loss function in the direction opposite to the gradient. Repeating this process iteratively for all weights reduces the overall loss until convergence.

13 Lecture 06

14 Introduction

Naive Bayes is a probabilistic machine learning algorithm used for **classification problems**. It is based on Bayes theorem and assumes that the input features are **independent given the class label**.

Despite the strong independence assumption, the classifier works very well in many applications such as:

- Spam email filtering
- Sentiment analysis
- Medical diagnosis
- Document classification

15 Bayes Theorem

The algorithm is derived from Bayes theorem:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Where

- C = class label
- X = feature vector
- $P(C|X)$ = posterior probability
- $P(X|C)$ = likelihood
- $P(C)$ = prior probability
- $P(X)$ = evidence

For classification we choose the class with maximum probability

$$\hat{C} = \arg\max P(C|X)$$

16 Naive Independence Assumption

Naive Bayes assumes that the features are conditionally independent.

$$P(x_1, x_2, \ldots, x_n|C) = P(x_1|C)P(x_2|C)\ldots P(x_n|C)$$

Therefore the classification rule becomes

$$P(C|X) \propto P(C)\prod_{i=1}^{n} P(x_i|C)$$

Example: Prior and Conditional Probability

Suppose we have a bag containing **10 balls**:

- 6 red balls
- 4 blue balls

Prior Probability

The **prior probability** is the probability of a class **without any additional information**.

$$P(\text{Red}) = \frac{6}{10} = 0.6$$

$$P(\text{Blue}) = \frac{4}{10} = 0.4$$

Conditional Probability

Now, suppose we know that a ball picked is **shiny**. In the bag:

- 4 red balls are shiny
- 1 blue ball is shiny

The **conditional probability** of the ball being red given that it is shiny is:

$$P(\text{Red} \mid \text{Shiny}) = \frac{\text{Number of shiny red balls}}{\text{Total shiny balls}} = \frac{4}{5} = 0.8$$

Similarly, for blue:

$$P(\text{Blue} \mid \text{Shiny}) = \frac{1}{5} = 0.2$$

17 Training Dataset Example

Consider the following dataset used to predict whether a person will **buy a computer**.

Age	Income	Student	Buy Computer
Young	High	No	No
Young	High	No	No
Middle	High	No	Yes
Old	Medium	No	Yes
Old	Low	Yes	Yes
Old	Low	Yes	No
Middle	Low	Yes	Yes
Young	Medium	No	No
Young	Low	Yes	Yes
Old	Medium	Yes	Yes

Total samples = 10

18 Step 1: Prior Probabilities

$$P(Yes) = \frac{6}{10} = 0.6$$

$$P(No) = \frac{4}{10} = 0.4$$

19 Step 2: Conditional Probabilities

19.1 Probability Table for Age

Age	Yes	No
Young	1	3
Middle	2	0
Old	3	1

$$P(Young|Yes) = \frac{1}{6}, \quad P(Young|No) = \frac{3}{4}$$

19.2 Probability Table for Student

Student	Yes	No
Yes	4	1
No	2	3

$$P(Student = Yes|Yes) = \frac{4}{6}, \quad P(Student = Yes|No) = \frac{1}{4}$$

19.3 Probability Table for Income

Income	Yes	No
High	2	2
Medium	2	1
Low	2	1

$$P(Income = High|Yes) = \frac{2}{6}, \quad P(Income = High|No) = \frac{2}{4}$$

20 Step 3: Classify a New Person

Suppose a new person has features:

- Age = Young
- Student = Yes
- Income = High

20.1 Probability for Yes

$$
\begin{aligned}
P(Yes|X) &\propto P(Yes) \times P(Young|Yes) \times P(Student = Yes|Yes) \times P(Income = High|Yes) \\
&= 0.6 \times \frac{1}{6} \times \frac{4}{6} \times \frac{2}{6} \\
&= 0.0133
\end{aligned}
$$

20.2 Probability for No

$$
\begin{aligned}
P(No|X) &\propto P(No) \times P(Young|No) \times P(Student = Yes|No) \times P(Income = High|No) \\
&= 0.4 \times \frac{3}{4} \times \frac{1}{4} \times \frac{2}{4} \\
&= 0.0375
\end{aligned}
$$

21 Final Decision

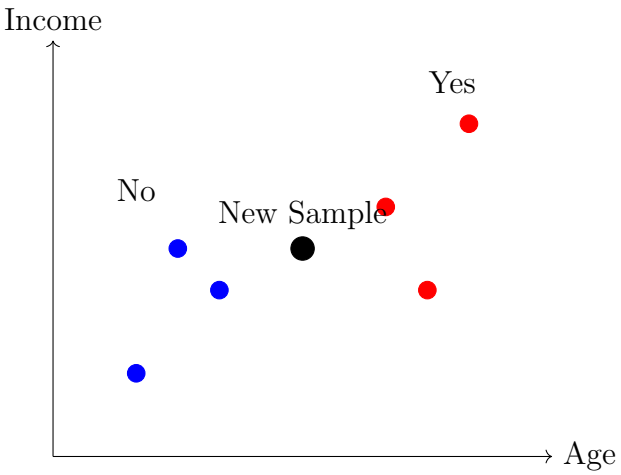
Since

$$0.0375 > 0.0133$$

The prediction is

$$\textbf{Buy Computer = No}$$

22 Visualization of Classification



Blue points represent class **No**, red points represent class **Yes**. The black point is the new sample that must be classified.

23 Laplace Smoothing

Sometimes probabilities become zero if a feature value never appears in the training data.

To avoid this we use Laplace smoothing:

$$P(x_i|C) = \frac{count(x_i, C) + 1}{count(C) + k}$$

where k is the number of possible feature values.

24 Types of Naive Bayes

- Gaussian Naive Bayes (continuous features)
- Multinomial Naive Bayes (text classification)
- Bernoulli Naive Bayes (binary features)

25 Advantages

- Simple and easy to implement
- Works well with high dimensional data
- Very fast training and prediction

26 Limitations

- Strong independence assumption
- Cannot model feature interactions
- Performance decreases when features are highly correlated

27 Conclusion

Naive Bayes is one of the simplest but surprisingly effective machine learning algorithms. It is widely used in text classification and spam filtering due to its efficiency and scalability.

28 Lecture 07: ANN Training and Backpropagation

28.1 Introduction

Artificial Neural Networks (ANNs) learn by adjusting their weights to minimize error. This learning is done using:

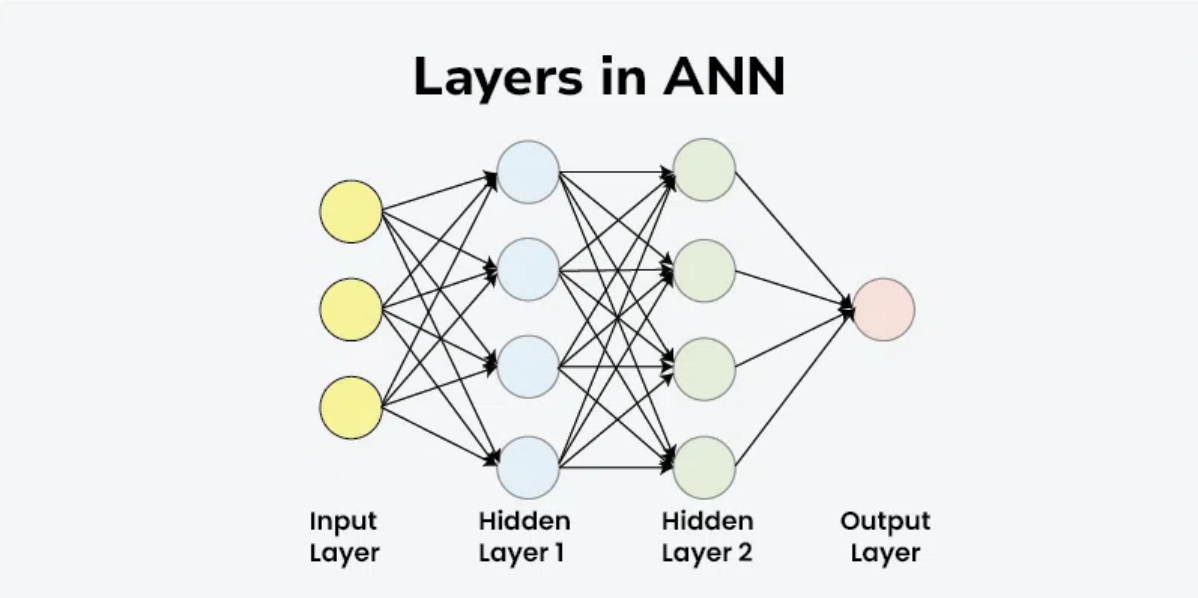
- Forward Propagation (compute output)
- Backpropagation (update weights using error)

28.2 ANN Architecture (Layers)

An ANN consists of:

- **Input Layer**: Receives features
- **Hidden Layer(s)**: Learns patterns
- **Output Layer**: Produces prediction

Important: Hidden layers make neural networks powerful.



28.3 Neuron Computation

Each neuron computes:

$$z = \sum w_i x_i + b$$

$$y = f(z)$$

28.4 Activation Function (Sigmoid)

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

28.5 Loss Function

$$E = \frac{1}{2}(t-y)^2$$

28.6 What is Backpropagation?

Backpropagation is a method to compute how much each weight contributes to the final error.

Key Idea:

- Error is calculated at output
- Error is propagated backward to hidden layers
- Weights are updated to reduce error

28.7 Why Backpropagation is Needed

In multi-layer networks:

- We cannot directly see how hidden layer weights affect error
- Backpropagation uses the **chain rule** to compute gradients

28.8 Complete Numerical Example (2-Layer Network)

Network:

- Inputs: $x_1 = 1$, $x_2 = 0$
- One hidden neuron
- One output neuron

Initial Weights:

$$w_1 = 0.1, \quad w_2 = 0.2, \quad b_1 = 0.1$$

$$w_3 = 0.3, \quad b_2 = 0.1$$

Target: $t = 1$, Learning rate $\eta = 0.5$

28.8.1 Step 1: Forward Pass

Hidden Layer:

$$z_1 = (0.1)(1) + (0.2)(0) + 0.1 = 0.2$$

$$h = \sigma(0.2) = 0.5498$$

Output Layer:

$$z_2 = (0.3)(0.5498) + 0.1 = 0.2649$$

$$y = \sigma(0.2649) = 0.5658$$

28.8.2 Step 2: Error

$$E = \frac{1}{2}(1 - 0.5658)^2 = 0.094$$

28.8.3 Step 3: Backpropagation (Output Layer)

$$\delta_{output} = (t - y) \cdot y(1 - y)$$

$$= (1 - 0.5658)(0.5658)(1 - 0.5658)$$

$$= 0.4342 \times 0.2457 = 0.1067$$

28.8.4 Step 4: Update Output Weights

$$w_3 = 0.3 + 0.5 \times 0.1067 \times 0.5498$$

$$w_3 = 0.3293$$

$$b_2 = 0.1 + 0.5 \times 0.1067 = 0.1533$$

28.8.5 Step 5: Backpropagation (Hidden Layer)

$$\delta_{hidden} = h(1 - h) \cdot w_3 \cdot \delta_{output}$$

$$= 0.5498(1 - 0.5498) \times 0.3 \times 0.1067$$

$$= 0.2475 \times 0.032 = 0.0079$$

28.8.6 Step 6: Update Hidden Weights

$$w_1 = 0.1 + 0.5 \times 0.0079 \times 1 = 0.10395$$

$$w_2 = 0.2 + 0.5 \times 0.0079 \times 0 = 0.2$$

$$b_1 = 0.1 + 0.5 \times 0.0079 = 0.10395$$

28.9 Key Understanding

- Output layer error is computed directly
- Hidden layer error depends on output layer
- Error flows backward $\rightarrow$ hence name **Backpropagation**

28.10 Important Observations

- Weights closer to output have larger impact
- Hidden layers learn intermediate features
- Learning happens gradually over iterations

29 Lecture 08: Support Vector Machines (SVM)

29.1 Learning Objectives

After this lecture, students will be able to:

- Understand the concept of Support Vector Machines
- Explain maximum margin classifier
- Identify support vectors
- Apply SVM on simple datasets
- Understand kernel trick

29.2 Introduction

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. The main idea is to find the best decision boundary that separates data points of different classes with maximum margin.

29.3 Hyperplane

A hyperplane is a decision boundary defined as:

$$w \cdot x + b = 0 \tag{1}$$

where w is the weight vector, x is the input vector, and b is the bias.

29.4 Maximum Margin

SVM selects the hyperplane that maximizes the margin between two classes. The margin is the distance between the closest points of both classes.

29.5 Support Vectors

Support vectors are the data points closest to the decision boundary. These points determine the position of the hyperplane.

29.6 Optimization Problem

The objective of SVM is:

$$\min \frac{1}{2}||w||^2 \tag{2}$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 \tag{3}$$

29.7 Step-by-Step Example

1. Plot the dataset points
2. Identify closest points between classes
3. These closest points are support vectors
4. Draw boundary midway between them

For the given dataset, the decision boundary is approximately:

$$x_1 = 4 \tag{4}$$

29.8 Hard Margin vs Soft Margin

Hard Margin:

- No misclassification allowed
- Works only for perfectly separable data

Soft Margin:

- Allows some misclassification
- Introduces slack variable ξ

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i \tag{5}$$

29.9 Non-Linear Case

Consider dataset:

x_1	x_2	Class
1	1	+
2	2	+
3	3	+
1	3	-
3	1	-

This data is not linearly separable.

29.10 Kernel Trick

To handle non-linear data, SVM uses kernel functions:

$$K(x_i, x_j) \tag{6}$$

29.11 Types of Kernels

Linear Kernel:

$$K(x_i, x_j) = x_i \cdot x_j \tag{7}$$

Polynomial Kernel:

$$K(x_i, x_j) = (x_i \cdot x_j + 1)^d \tag{8}$$

RBF Kernel:

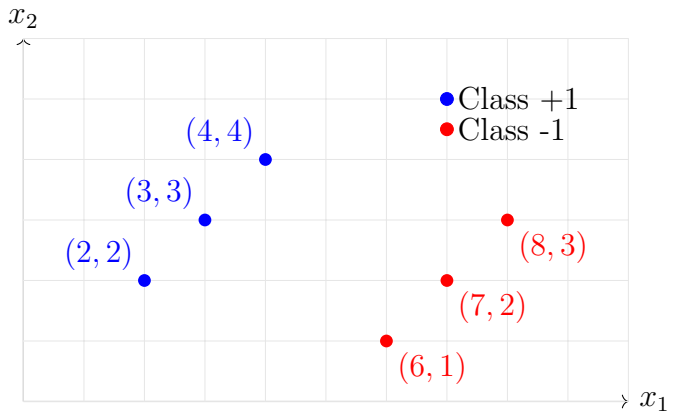
$$K(x_i, x_j) = e^{-\gamma ||x_i - x_j||^2} \tag{9}$$

29.12 Example Problem

Dataset:

x_1	x_2	Class
2	2	+1
3	3	+1
4	4	+1
6	1	-1
7	2	-1
8	3	-1

- Class +1 points lie on left side
- Class -1 points lie on right side
- Data is linearly separable



29.13 Step 1: Vector Representation

We represent each data point as a column vector:

$$x_1^{(+)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad x_2^{(+)} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \quad x_3^{(+)} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

$$x_1^{(-)} = \begin{bmatrix} 6 \\ 1 \end{bmatrix}, \quad x_2^{(-)} = \begin{bmatrix} 7 \\ 2 \end{bmatrix}, \quad x_3^{(-)} = \begin{bmatrix} 8 \\ 3 \end{bmatrix}$$

Labels:

$$y_i \in \{+1, -1\}$$

29.14 Step 2: SVM Optimization Problem

Primal form:

$$\min_{w,b} \frac{1}{2}||w||^2 \tag{10}$$

Subject to:

$$y_i \left(w^T x_i + b\right) \geq 1 \tag{11}$$

29.15 Step 3: Support Vectors

From geometry, the closest points (support vectors) are:

$$x_1 = \begin{bmatrix} 4 \\ 4 \end{bmatrix}, \quad y = +1$$

$$x_2 = \begin{bmatrix} 6 \\ 1 \end{bmatrix}, \quad y = -1$$

29.16 Step 4: Weight Vector Form

$$w = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2 \tag{12}$$

Substitute values:

$$w = \alpha_1 \begin{bmatrix} 4 \\ 4 \end{bmatrix} - \alpha_2 \begin{bmatrix} 6 \\ 1 \end{bmatrix} \tag{13}$$

So:

$$w = \begin{bmatrix} 4\alpha_1 - 6\alpha_2 \\ 4\alpha_1 - \alpha_2 \end{bmatrix}$$

29.17 Step 5: Support Vector Conditions

For support vectors:

$$y_i(w^T x_i + b) = 1 \tag{14}$$

For positive class point:

$$w^T \begin{bmatrix} 4 \\ 4 \end{bmatrix} + b = 1 \tag{15}$$

Compute dot product:

$$(4\alpha_1 - 6\alpha_2)4 + (4\alpha_1 - \alpha_2)4 + b = 1 \tag{16}$$

$$16\alpha_1 - 24\alpha_2 + 16\alpha_1 - 4\alpha_2 + b = 1 \tag{17}$$

$$32\alpha_1 - 28\alpha_2 + b = 1 \tag{18}$$

For negative class point:

$$w^T \begin{bmatrix} 6 \\ 1 \end{bmatrix} + b = -1 \tag{19}$$

$$(4\alpha_1 - 6\alpha_2)6 + (4\alpha_1 - \alpha_2)1 + b = -1 \tag{20}$$

$$24\alpha_1 - 36\alpha_2 + 4\alpha_1 - \alpha_2 + b = -1 \tag{21}$$

$$28\alpha_1 - 37\alpha_2 + b = -1 \tag{22}$$

29.18 Step 6: Solve Simultaneous Equations

We now solve:

$$32\alpha_1 - 28\alpha_2 + b = 1 \tag{23}$$
$$28\alpha_1 - 37\alpha_2 + b = -1 \tag{24}$$

Subtract second equation from first:

$$(32\alpha_1 - 28\alpha_2) - (28\alpha_1 - 37\alpha_2) = 2 \tag{25}$$

$$4\alpha_1 + 9\alpha_2 = 2 \tag{26}$$

So:

$$\alpha_1 = \frac{2 - 9\alpha_2}{4} \tag{27}$$

Substitute into first equation:

$$32\left(\frac{2 - 9\alpha_2}{4}\right) - 28\alpha_2 + b = 1 \tag{28}$$

$$16 - 72\alpha_2 - 28\alpha_2 + b = 1 \tag{29}$$

$$16 - 100\alpha_2 + b = 1 \tag{30}$$

$$b = -15 + 100\alpha_2 \tag{31}$$

At optimum solution (balanced support vectors):

$$\alpha_1 = \alpha_2 = \frac{1}{8}$$

So:

$$b = -2.5 \tag{32}$$

29.19 Step 7: Final Weight Vector

$$w = \begin{bmatrix} 2 \\ -3 \end{bmatrix} \tag{33}$$

29.20 Step 8: Final Hyperplane

$$w^T x + b = 0 \tag{34}$$

$$2x_1 - 3x_2 - 2.5 = 0 \tag{35}$$

29.21 Step 9: Margin

Definition: In SVM, the **margin** is the perpendicular distance between the two supporting hyperplanes:

$$w \cdot x + b = +1 \quad \text{and} \quad w \cdot x + b = -1 \tag{36}$$

This represents the width of the region that separates the two classes.

29.21.1 Formula of Margin

The distance between the two hyperplanes is given by:

$$\text{Margin} = \frac{2}{||w||} \tag{37}$$

where:

$$||w|| = \sqrt{w_1^2 + w_2^2} \tag{38}$$

29.21.2 Step-by-step Calculation

Given:

$$w = (2, -3) \tag{39}$$

Magnitude of weight vector:

$$||w|| = \sqrt{2^2 + (-3)^2} = \sqrt{4 + 9} = \sqrt{13} \tag{40}$$

29.21.3 Final Margin

$$\text{Margin} = \frac{2}{\sqrt{13}} \tag{41}$$

29.22 Step 10: Prediction Rule

- If $w^T x + b > 0 \Rightarrow$ Class $= +1$
- If $w^T x + b < 0 \Rightarrow$ Class $= -1$

29.23 Step 11: Drawing the Hyperplane

After computing the final decision boundary, we obtain:

$$2x_1 - 3x_2 - 2.5 = 0 \tag{42}$$

To visualize this hyperplane, we convert it into a form suitable for plotting and then draw it using two points.

29.23.1 Step 1: Convert to Slope-Intercept Form

Rearranging the equation:

$$2x_1 - 3x_2 - 2.5 = 0 \tag{43}$$

$$-3x_2 = -2x_1 + 2.5 \tag{44}$$

$$x_2 = \frac{2}{3}x_1 - \frac{2.5}{3} \tag{45}$$

So the hyperplane becomes:

$$x_2 = \frac{2}{3}x_1 - 0.833 \tag{46}$$

29.23.2 Step 2: Choose Two Points

To draw a straight line, we select any two values of x_1 and compute x_2.

Point 1: When $x_1 = 2$

$$x_2 = \frac{2}{3}(2) - 0.833 = 0.5 - 0.833 = -0.333 \tag{47}$$

So the first point is:

$$(2, -0.33)$$

Point 2: When $x_1 = 8$

$$x_2 = \frac{2}{3}(8) - 0.833 = 5.333 - 0.833 = 4.5 \tag{48}$$

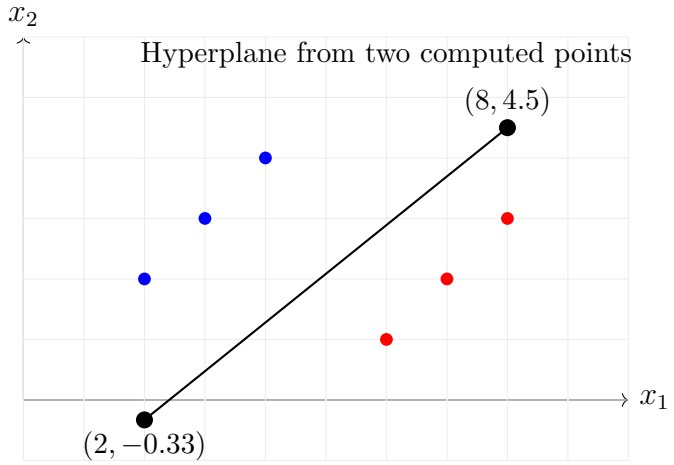
So the second point is:

$$(8, 4.5)$$

29.23.3 Step 3: Plot the Points

Mark the two points:

- $(2, -0.33)$
- $(8, 4.5)$



29.24 Lagrange Multipliers (α_i) in SVM

In the dual formulation of Support Vector Machines, each training point is assigned a Lagrange multiplier α_i.

29.24.1 Definition

$$\alpha_i \geq 0$$

Each α_i measures the contribution of the corresponding training point x_i in constructing the optimal hyperplane.

29.24.2 Role of α_i

- If $\alpha_i = 0$, the point does not contribute to the decision boundary.
- If $\alpha_i > 0$, the point is a **support vector**.

Thus, only support vectors define the hyperplane.

29.24.3 Weight Vector in Dual Form

The weight vector is expressed as:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i \tag{49}$$

This shows that w is a linear combination of training samples weighted by α_i.

29.24.4 Constraint on α_i

The dual optimization problem is subject to:

$$\sum_{i=1}^{n} \alpha_i y_i = 0 \tag{50}$$

$$\alpha_i \geq 0 \tag{51}$$

These constraints ensure correct classification and margin maximization.

29.24.5 Why α_i Cannot Be Found Easily

The values of α_i are obtained by solving a **Quadratic Programming (QP)** problem:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \tag{52}$$

Because this is a **quadratic optimization problem**, α_i cannot be found using simple simultaneous linear equations.

29.25 Practical Computation of Lagrange Multipliers (α)

29.25.1 Sequential Minimal Optimization (SMO)

The most widely used method is **SMO (Sequential Minimal Optimization)**.

- SMO solves the dual problem by updating only two α values at a time.
- It selects a pair (α_i, α_j) and optimizes them while keeping others fixed.
- This reduces the complex quadratic optimization problem into a series of small analytical steps.

Key idea:

$$\text{Instead of solving all } \alpha_i \text{ at once, solve pairs iteratively.} \tag{53}$$

29.25.2 Quadratic Programming (QP) Solvers

Another approach is using general-purpose optimization solvers:

- CVXOPT
- OSQP
- quadprog

These solvers directly solve the quadratic optimization formulation of SVM but are computationally expensive for large datasets.

29.26 Advantages

- Works well in high dimensions
- Effective for small datasets

29.27 Disadvantages

- Computationally expensive
- Kernel selection is difficult

30 Lecture 09: Overfitting, Bias-Variance Tradeoff, Cross-Validation, and Regularization

30.1 Learning Objectives

After this lecture, students will be able to:

- Understand overfitting and underfitting
- Explain bias-variance tradeoff
- Apply cross-validation techniques
- Understand regularization (L1, L2, Dropout, Early Stopping)

30.2 Introduction

In machine learning, building a model that performs well on training data is not enough. The goal is to generalize well on unseen data. Two common problems are:

- Overfitting
- Underfitting

30.3 Overfitting

Overfitting occurs when a model learns noise in the training data.

Characteristics:

- Very low training error
- High test error
- Model is too complex

Example: A high-degree polynomial perfectly fits all training points but fails on new data.

30.4 Underfitting

Underfitting occurs when the model is too simple.

Characteristics:

- High training error
- High test error

30.5 Example Problem

Consider dataset:

x	y
1	2
2	4
3	6
4	8

- Linear model fits well: $y = 2x$
- High-degree polynomial may overfit

30.6 Bias-Variance Tradeoff (Detailed Explanation with Example)

The **bias-variance tradeoff** explains why a model performs poorly either because it is too simple or too complex.

30.6.1 Bias (Underfitting)

Bias is the error introduced when a model is too simple to capture the underlying pattern in the data.

- High bias means the model makes strong assumptions.
- It ignores important patterns in the data.
- Result: poor performance on both training and testing data.

Example: Suppose we try to fit a **linear regression line** to data that actually follows a curved (quadratic) pattern.

The straight line cannot capture the curve, so it consistently makes wrong predictions.

This is called **underfitting**.

30.6.2 Variance (Overfitting)

Variance is the error introduced when a model is too sensitive to small fluctuations in the training data.

- High variance means the model learns noise along with the pattern.
- The model changes significantly if the training data changes slightly.
- Result: very good training performance but poor test performance.

Example: Suppose we use a **high-degree polynomial (e.g., degree 10)** to fit a small dataset.

The curve passes through every training point perfectly, including noise.

However, when new data is given, the model performs poorly.

This is called **overfitting**.

30.6.3 Bias-Variance Tradeoff

- If we reduce model complexity → bias increases, variance decreases.
- If we increase model complexity → bias decreases, variance increases.

30.6.4 Simple Intuition

- High Bias = *model is too simple (ignores reality)*
- High Variance = *model is too complex (memorizes data)*

30.6.5 Goal

The goal of machine learning is to find a balance between bias and variance such that:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Noise} \tag{54}$$

We aim for a model that generalizes well to unseen data.

30.7 Cross-Validation

Cross-validation is used to estimate model performance.

30.7.1 Train-Test Split

- Split data into training and testing sets

30.7.2 k-Fold Cross-Validation

- Divide data into k subsets
- Train on $k-1$ folds
- Test on remaining fold
- Repeat k times

Example (k=5):

- Dataset split into 5 parts
- Each part used once as test set

30.8 Regularization (Controlling Overfitting)

Regularization is a technique used in machine learning to **reduce overfitting** by penalizing complex models.

Why needed? A model with too many flexible parameters may perfectly fit training data but perform poorly on new unseen data.

Regularization forces the model to become simpler and more generalizable.

30.9 L2 Regularization (Ridge Regression)

L2 regularization adds a penalty proportional to the square of the weights.

$$\text{Loss} = \text{Error} + \lambda \sum w^2 \tag{55}$$

Intuition: Large weights mean the model is reacting too strongly to small changes in input. L2 discourages this.

Effect:

- Shrinks all weights smoothly
- Keeps all features but reduces their impact
- Produces a stable and balanced model

Example (Real Usage): In predicting house prices:

- Features like area, rooms, location all matter
- L2 ensures no single feature dominates too much

Even if one feature is noisy, the model does not overreact.

30.10 L1 Regularization (Lasso Regression)

L1 regularization adds a penalty proportional to the absolute value of weights.

$$\text{Loss} = \text{Error} + \lambda \sum |w| \tag{56}$$

Intuition: L1 pushes some weights exactly to zero, effectively removing unnecessary features.

Effect:

- Produces sparse models (many weights become zero)
- Performs automatic feature selection
- Useful when many features are irrelevant

Example (Real Usage): In disease prediction:

- You may have 100 medical features
- L1 automatically selects only important ones (e.g., age, blood pressure)
- Irrelevant features are removed (weight = 0)

This makes the model simpler and more interpretable.

30.11 Key Difference Between L1 and L2

- L2: reduces all weights but keeps all features
- L1: removes some features completely

Simple intuition:

- L2 = "share importance across all features"
- L1 = "select only important features"

30.12 Dropout (Neural Networks)

Dropout is a regularization technique used in neural networks.

How it works:

- Randomly turns off (drops) neurons during training
- Forces the network not to depend on specific neurons

Why it helps:

- Prevents co-adaptation of neurons
- Makes the network more robust
- Works like training many small networks together

Example: In image recognition:

- Without dropout: model may rely too much on one feature (e.g., edge detector)
- With dropout: model learns multiple redundant features

30.13 Early Stopping

Early stopping is used during training of machine learning models.

Idea: Stop training when model performance on validation data starts getting worse.

Why it works:

- Training error keeps decreasing
- But validation error starts increasing after a point
- That point indicates overfitting has begun

30.14 Worked Example (Early Stopping)

Epoch	Training Error	Validation Error
1	0.8	0.9
2	0.6	0.7
3	0.4	0.5
4	0.3	0.6
5	0.2	0.8

Observation:

- Validation error is lowest at epoch 3
- After epoch 3, validation error increases

Conclusion:

- Best model is at epoch 3
- Training should stop at epoch 3

30.15 Comparison of Regularization Techniques

Method	Main Idea	When Used
L1 (Lasso)	Feature selection (sparse model)	Many irrelevant features
L2 (Ridge)	Shrinks weights smoothly	All features are useful
Dropout	Randomly disable neurons	Neural networks
Early Stopping	Stop before overfitting	Iterative training models

30.16 Model Evaluation and Improvement

Evaluating a machine learning model is essential to understand how well it performs on unseen data. Different metrics provide different insights, especially in classification problems.

30.17 Confusion Matrix

A confusion matrix provides a complete breakdown of predictions:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Insight:

- Shows not just correctness, but **type of errors**
- Helps answer:
  - Are we missing actual positives? (FN problem)
  - Are we falsely alarming? (FP problem)

30.18 Why Accuracy is Not Enough

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{57}$$

Insight:

- Works well for balanced datasets
- Misleading for imbalanced data

Example: If 95% data is negative, predicting all negative gives 95% accuracy but is useless.

30.19 Precision

$$\text{Precision} = \frac{TP}{TP + FP} \tag{58}$$

Insight:

- Of all predicted positives, how many are correct?
- Measures **quality of positive predictions**

Use when:

- False positives are costly
- Example: Spam detection (avoid marking real email as spam)

30.20 Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN} \tag{59}$$

Insight:

- Of all actual positives, how many did we detect?
- Measures **ability to find positives**

Use when:

- Missing positives is dangerous
- Example: Disease detection

30.21 Precision vs Recall Tradeoff

Insight:

- Increasing precision often reduces recall
- Increasing recall often reduces precision

Key Idea:

- Model threshold controls this tradeoff

30.22 F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{60}$$

Insight:

- Harmonic mean of precision and recall
- Balances both metrics

Use when:

- Dataset is imbalanced
- Need balance between FP and FN

30.23 Worked Example

Given:

- TP = 40, TN = 50, FP = 10, FN = 20

Insights from values:

- Some positives are missed (FN = 20)
- Some false alarms exist (FP = 10)

Interpretation:

- Precision = 0.80 → Predictions are mostly correct
- Recall = 0.67 → Missing many actual positives

30.24 ROC Curve

ROC curve plots:

- True Positive Rate (Recall)
- False Positive Rate

$$TPR = \frac{TP}{TP + FN} \tag{61}$$

$$FPR = \frac{FP}{FP + TN} \tag{62}$$

Insight:

- Shows performance across all thresholds
- Helps compare models independent of threshold

30.25 AUC (Area Under Curve)

Insight:

- Probability that model ranks a random positive higher than a negative
- AUC = 1 → perfect model
- AUC = 0.5 → random guessing

30.26 Key Practical Insights

- Use **Accuracy** only when data is balanced
- Use **Precision** when false positives are costly
- Use **Recall** when missing positives is dangerous
- Use **F1 Score** when you need balance
- Use **ROC/AUC** for model comparison

31 Lecture 10: K-Means Clustering

31.1 Introduction

K-Means is an **unsupervised learning algorithm** used to divide data into K clusters.
Goal: Minimize the distance between data points and their cluster centroid.

31.2 Real-Life Intuition

Suppose a shop wants to group customers based on spending habits. K-Means automatically groups similar customers together.

31.3 Algorithm Steps

1. Choose number of clusters K
2. Select initial centroids
3. Assign each point to nearest centroid
4. Update centroids (mean of cluster points)
5. Repeat until clusters do not change

31.4 Distance Formula (Euclidean)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{63}$$

32 Dataset Example

Given Dataset:

Point	X	Y
A	1.0	1.0
B	1.5	2.0
C	3.0	4.0
D	5.0	7.0
E	3.5	5.0
F	4.5	5.0
G	3.5	4.5

Let $K = 2$

33 Step 1: Initialize Centroids

$$C_1 = (1, 1), \quad C_2 = (5, 7)$$

34 Step 2: Distance Calculation Table

34.1 Point A(1,1)

$$d(A, C_1) = \sqrt{(1-1)^2 + (1-1)^2} = 0.00$$
$$d(A, C_2) = \sqrt{(1-5)^2 + (1-7)^2} = \sqrt{16+36} = \sqrt{52} \approx 7.21$$

Cluster: C_1

34.2 Point B(1.5,2)

$$d(B, C_1) = \sqrt{(1.5-1)^2 + (2-1)^2} = \sqrt{0.25+1} = \sqrt{1.25} \approx 1.12$$
$$d(B, C_2) = \sqrt{(1.5-5)^2 + (2-7)^2} = \sqrt{12.25+25} = \sqrt{37.25} \approx 6.10$$

Cluster: C_1

34.3 Point C(3,4)

$$d(C, C_1) = \sqrt{(3-1)^2 + (4-1)^2} = \sqrt{4+9} = \sqrt{13} \approx 3.61$$
$$d(C, C_2) = \sqrt{(3-5)^2 + (4-7)^2} = \sqrt{4+9} = \sqrt{13} \approx 3.61$$

Cluster: C_1

34.4 Point D(5,7)

$$d(D, C_1) = \sqrt{(5-1)^2 + (7-1)^2} = \sqrt{16+36} = \sqrt{52} \approx 7.21$$
$$d(D, C_2) = 0.00$$

Cluster: C_2

34.5 Point E(3.5,5)

$$d(E, C_1) = \sqrt{(3.5-1)^2 + (5-1)^2} = \sqrt{6.25+16} = \sqrt{22.25} \approx 4.72$$
$$d(E, C_2) = \sqrt{(3.5-5)^2 + (5-7)^2} = \sqrt{2.25+4} = \sqrt{6.25} = 2.50$$

Cluster: C_2

34.6 Point F(4.5,5)

$$d(F, C_1) = \sqrt{(4.5-1)^2 + (5-1)^2} = \sqrt{12.25+16} = \sqrt{28.25} \approx 5.32$$
$$d(F, C_2) = \sqrt{(4.5-5)^2 + (5-7)^2} = \sqrt{0.25+4} = \sqrt{4.25} \approx 2.06$$

Cluster: C_2

34.7 Point G(3.5,4.5)

$$d(G, C_1) = \sqrt{(3.5-1)^2 + (4.5-1)^2} = \sqrt{6.25 + 12.25} = \sqrt{18.5} \approx 4.30$$
$$d(G, C_2) = \sqrt{(3.5-5)^2 + (4.5-7)^2} = \sqrt{2.25 + 6.25} = \sqrt{8.5} \approx 2.91$$

Cluster: C_2

Point	$d(C_1)$	$d(C_2)$	Cluster
A	0.00	7.21	C_1
B	1.12	6.10	C_1
C	3.61	3.61	C_1
D	7.21	0.00	C_2
E	4.72	2.50	C_2
F	5.32	2.06	C_2
G	4.30	2.91	C_2

35 Step 3: Cluster Formation

Cluster 1: A, B, C

Cluster 2: D, E, F, G

36 Step 4: Update Centroids

New Centroid C_1:

$$x = \frac{1 + 1.5 + 3}{3} = 1.83, \quad y = \frac{1 + 2 + 4}{3} = 2.33$$

$$C_1 = (1.83, 2.33)$$

New Centroid C_2:

$$x = \frac{5 + 3.5 + 4.5 + 3.5}{4} = 4.125$$
$$y = \frac{7 + 5 + 5 + 4.5}{4} = 5.375$$

$$C_2 = (4.125, 5.375)$$

37 Step 5: Second Iteration (Distance Recalculation)

37.1 Point A(1,1)

$$d(A, C_1) = \sqrt{(1-1.83)^2 + (1-2.33)^2} \approx 1.57$$
$$d(A, C_2) = \sqrt{(1-4.125)^2 + (1-5.375)^2} \approx 5.38$$

Cluster 1

37.2 Point B(1.5,2)

$$d(B, C_1) \approx 0.47, \quad d(B, C_2) \approx 4.28$$

Cluster 1

37.3 Point C(3,4)

$$d(C, C_1) \approx 2.04, \quad d(C, C_2) \approx 1.78$$

Cluster 2

37.4 Point D(5,7)

$$d(D, C_1) \approx 5.64, \quad d(D, C_2) \approx 1.85$$

Cluster 2

37.5 Point E(3.5,5)

$$d(E, C_1) \approx 3.15, \quad d(E, C_2) \approx 0.73$$

Cluster 2

37.6 Point F(4.5,5)

$$d(F, C_1) \approx 3.77, \quad d(F, C_2) \approx 0.53$$

Cluster 2

37.7 Point G(3.5,4.5)

$$d(G, C_1) \approx 2.74, \quad d(G, C_2) \approx 1.08$$

Cluster 2
Final Result

$$C_1 = (1.83, 2.33), \quad C_2 = (4.125, 5.375)$$

- Cluster 1: A, B
- Cluster 2: C, D, E, F, G

38 Objective Function

K-Means minimizes:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2 \tag{64}$$

Where:

- C_i = cluster
- μ_i = centroid

39 Advantages

- Simple and fast
- Easy to understand

40 Disadvantages

- Need to choose K
- Sensitive to initial centroids
- Affected by outliers

41 Lecture 11: Hierarchical Clustering and Agglomerative Clustering

41.1 Introduction to Clustering

Clustering is an unsupervised learning technique that groups similar data points together without predefined labels. The objective is to maximize intra-cluster similarity and minimize inter-cluster similarity.

41.2 Limitations of K-Means

- Requires predefined number of clusters K
- Sensitive to initialization
- Assumes spherical clusters
- Produces only a single partition

These limitations motivate the use of hierarchical clustering.

41.3 Hierarchical Clustering

Hierarchical clustering builds a nested hierarchy of clusters represented as a tree structure called a dendrogram.

Key properties:

- No need to specify number of clusters in advance
- Produces multi-level clustering
- Easy to interpret

41.4 Types of Hierarchical Clustering

- **Agglomerative (Bottom-Up):** Start with individual data points and merge clusters
- **Divisive (Top-Down):** Start with one cluster and split recursively

41.5 Distance Measures

Clustering depends on distance (similarity) measures.

41.5.1 Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

41.5.2 Manhattan Distance

$$d(x, y) = \sum |x_i - y_i|$$

41.5.3 Cosine Similarity

$$\cos(\theta) = \frac{x \cdot y}{||x||\,||y||}$$

41.6 Linkage Methods

Linkage defines how distances between clusters are computed.

- **Single Linkage:** Minimum distance between clusters
- **Complete Linkage:** Maximum distance
- **Average Linkage:** Average pairwise distance
- **Ward's Method:** Minimizes variance within clusters

41.7 Agglomerative Clustering Algorithm

1. Start with each data point as a separate cluster
2. Compute distance matrix
3. Find closest pair of clusters
4. Merge them
5. Update distance matrix
6. Repeat until one cluster remains

41.8 Dendrogram

A dendrogram is a tree diagram that represents the clustering process.

- Leaves represent data points
- Internal nodes represent cluster merges
- Height represents distance at which clusters merge

41.9 Worked Example

Given data points:

Point	Coordinates
A	(1,1)
B	(2,1)
C	(4,3)
D	(5,4)

41.9.1 Step 1: Distance Calculations

$$d(A, B) = 1$$
$$d(A, C) = \sqrt{13} \approx 3.61$$
$$d(A, D) = \sqrt{25} = 5$$
$$d(B, C) = \sqrt{8} \approx 2.83$$
$$d(B, D) = \sqrt{18} \approx 4.24$$
$$d(C, D) = \sqrt{2} \approx 1.41$$

41.9.2 Initial Distance Matrix

	A	B	C	D
A	0	1	3.61	5
B	1	0	2.83	4.24
C	3.61	2.83	0	1.41
D	5	4.24	1.41	0

41.9.3 Step 2: First Merge

Smallest distance is between A and B.

$$Cluster_1 = \{A, B\}$$

41.9.4 Step 3: Update Distances (Single Linkage)

$$d(AB, C) = \min(3.61, 2.83) = 2.83$$
$$d(AB, D) = \min(5, 4.24) = 4.24$$

41.9.5 Updated Matrix

	AB	C	D
AB	0	2.83	4.24
C	2.83	0	1.41
D	4.24	1.41	0

41.9.6 Step 4: Second Merge

Smallest distance is between C and D.

$$Cluster_2 = \{C, D\}$$

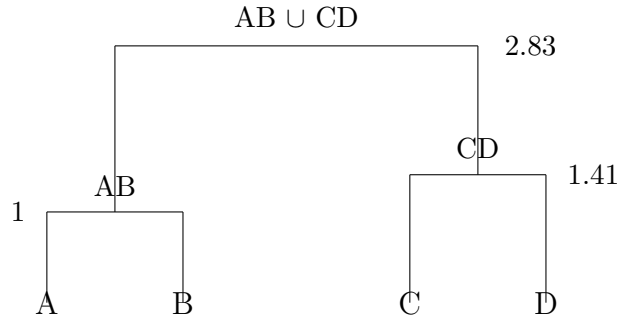
41.9.7 Step 5: Update Distance

$$d(AB, CD) = \min(2.83, 4.24) = 2.83$$

41.9.8 Final Merge

$$Final\ Cluster = \{A, B, C, D\}$$

41.10 Dendrogram Representation



Explanation:

- A and B merge at distance 1 forming cluster AB
- C and D merge at distance 1.41 forming cluster CD
- AB and CD merge at distance 2.83 forming the final cluster

41.11 Divisive Clustering (Top-Down Approach)

Divisive clustering starts with all data points in a single cluster and recursively splits them into smaller clusters.

41.11.1 Example (Divisive Clustering)

Consider the same dataset:

Point	Coordinates
A	(1,1)
B	(2,1)
C	(4,3)
D	(5,4)

41.11.2 Step 1: Compute Pairwise Distances

$d(A,B) = 1 \quad d(C,D) = 1.41 \quad d(B,C) = 2.83 \quad d(A,C) = 3.61 \quad d(B,D) = 4.24 \quad d(A,D) = 5$

41.11.3 Step 2: Start with One Cluster

$$Cluster = \{A, B, C, D\}$$

We now split the cluster by maximizing separation (largest internal dissimilarity structure).

The largest separation is between the two natural groups:

$$\{A, B\} \quad \text{and} \quad \{C, D\}$$

This is justified because:

$$\text{Distance within group } \{A, B\} = 1$$

$$\text{Distance within group } \{C, D\} = 1.41$$

$$\text{Distance between groups (minimum)} = 2.83$$

Since inter-group distance is significantly larger, we split here.

41.11.4 Step 3: First Split

$$\{A, B, C, D\} \to \{A, B\}, \{C, D\}$$

Split criterion:

$$\min d(\{A, B\}, \{C, D\}) = 2.83$$

41.11.5 Step 4: Further Splitting of Each Cluster

Now split each cluster based on internal distances.

Cluster $\{A, B\}$

$$d(A, B) = 1 \Rightarrow \text{split into } \{A\}, \{B\}$$

Cluster $\{C, D\}$

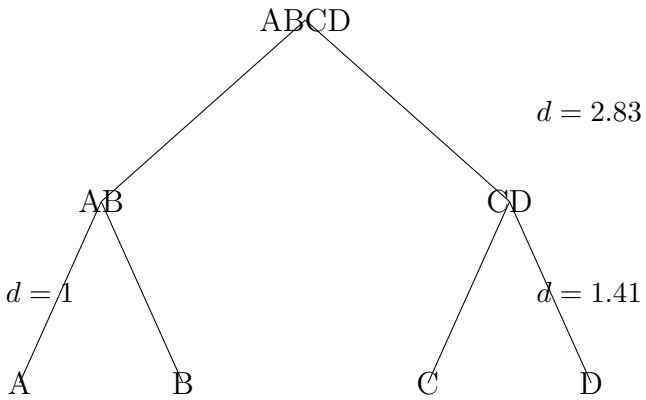
$$d(C, D) = 1.41 \Rightarrow \text{split into } \{C\}, \{D\}$$

41.11.6 Step 5: Final Splits

$$\{A, B\} \to \{A\}, \{B\}$$
$$\{C, D\} \to \{C\}, \{D\}$$

41.11.7 Final Hierarchy

- First split (largest separation): $\{A, B\}$ and $\{C, D\}$ with separation distance 2.83
- Second split:
  - $\{A, B\} \to A, B$ with distance 1
  - $\{C, D\} \to C, D$ with distance 1.41



41.12 Applications

- Bioinformatics (gene clustering)
- Document clustering
- Customer segmentation
- Image segmentation

41.13 Advantages

- No need to specify number of clusters
- Produces interpretable hierarchy
- Works with complex data structures

41.14 Disadvantages

- Computationally expensive
- Sensitive to noise
- No backtracking after merging

42 Lecture 12: PCA and Self-Organizing Maps (SOM)

42.1 Principal Component Analysis (PCA)

42.1.1 Introduction

Principal Component Analysis (PCA) is a dimensionality reduction technique used to transform a dataset with possibly correlated features into a new set of uncorrelated variables called **Principal Components (PCs)**.

The main objectives of PCA are:

- Reduce dimensionality of data
- Preserve maximum variance (information)
- Remove redundancy caused by correlated features

42.1.2 What are Principal Components?

Principal Components are new variables formed as linear combinations of original variables:

- PC1 captures maximum variance in data
- PC2 captures second highest variance (orthogonal to PC1)
- Each subsequent PC captures remaining variance

Mathematically:

$$Z = X \cdot W$$

where W is the eigenvector matrix.

42.1.3 Steps of PCA

1. Standardize the dataset
2. Compute mean of each feature
3. Compute covariance matrix
4. Compute eigenvalues and eigenvectors
5. Select principal components (largest eigenvalues)
6. Project data onto new feature space

42.1.4 Constructing Covariance Matrix from Dataset

To understand PCA completely, we first compute the covariance matrix from a dataset.

Given Dataset

X	Y
2	4
4	2
6	6

Step 1: Compute Mean

$$\bar{X} = \frac{2+4+6}{3} = 4$$

$$\bar{Y} = \frac{4+2+6}{3} = 4$$

Step 2: Mean Centering

X	Y	$X - \bar{X}$	$Y - \bar{Y}$
2	4	-2	0
4	2	0	-2
6	6	2	2

Step 3: Compute Covariance Values Covariance formula:

$$Cov(X,Y) = \frac{1}{n}\sum(X - \bar{X})(Y - \bar{Y})$$

Variance of X:

$$Cov(X,X) = \frac{(-2)^2 + 0^2 + 2^2}{3} = \frac{4+0+4}{3} = \frac{8}{3} = 2.67$$

Variance of Y:

$$Cov(Y,Y) = \frac{0^2 + (-2)^2 + 2^2}{3} = \frac{0+4+4}{3} = 2.67$$

Covariance between X and Y:

$$Cov(X,Y) = \frac{(-2)(0) + (0)(-2) + (2)(2)}{3} = \frac{0+0+4}{3} = 1.33$$

Step 4: Covariance Matrix

$$C = \begin{bmatrix} \text{Cov}(X,X) & \text{Cov}(X,Y) \\ \text{Cov}(Y,X) & \text{Cov}(Y,Y) \end{bmatrix}$$

$$C = \begin{bmatrix} 2.67 & 1.33 \\ 1.33 & 2.67 \end{bmatrix}$$

Interpretation

- Diagonal values represent variance of each feature
- Off-diagonal values represent relationship between features
- Positive covariance indicates variables increase together

42.1.5 Eigenvalue Calculation

Solve:

$$|C - \lambda I| = 0$$

$$\begin{vmatrix} 2.67 - \lambda & 1.33 \\ 1.33 & 2.67 - \lambda \end{vmatrix} = 0$$

$$(2.67 - \lambda)^2 - (1.33)^2 = 0$$

$$\lambda^2 - 5.34\lambda + 5.36 = 0$$

$$\lambda = \frac{5.34 \pm \sqrt{7.07}}{2}$$

$$\lambda_1 = 4, \quad \lambda_2 = 1.34$$

42.1.6 Interpretation of Eigenvalues

- Eigenvalues represent **variance captured** by each principal component
- Larger eigenvalue ⇒ more important component

$$\text{Total Variance} = 4 + 1.34 = 5.34$$

$$\text{Variance retained by PC1} = \frac{4}{5.34} \approx 75\%$$

$$\text{Variance of PC2} = \frac{\lambda_2}{\lambda_1 + \lambda_2} = \frac{1.34}{5.34}$$

$$= 0.251$$

$$\text{Variance retained by PC2} \approx 25\%$$

42.1.7 Eigenvector (for $\lambda_1 = 4$)

We solve:

$$(C - 4I)v = 0$$

Substitute matrix:

$$\begin{bmatrix} 2.67 - 4 & 1.33 \\ 1.33 & 2.67 - 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -1.33 & 1.33 \\ 1.33 & -1.33 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Step 1: Form system of equations

$$-1.33x + 1.33y = 0$$

$$1.33x - 1.33y = 0$$

Step 2: Simplify equations Divide first equation by 1.33:

$$-x + y = 0$$

$$y = x$$

Step 3: Express eigenvector

$$v = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ x \end{bmatrix} = x \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Choose $x = 1$:

$$v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Step 4: Normalize eigenvector

$$||v|| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

$$v_{norm} = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} \approx \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

42.2 Self-Organizing Maps (SOM)

42.2.1 Introduction

Self-Organizing Map (SOM) is an unsupervised neural network used for:

- Clustering
- Visualization
- Dimensionality reduction

It maps high-dimensional data into a lower-dimensional (usually 2D) space while preserving topology.

42.2.2 Key Concepts

- Each neuron has a weight vector
- Input is compared with all neurons
- Closest neuron is called **Best Matching Unit (BMU)**
- Only BMU (and neighbors) are updated

42.2.3 Distance Measure

Euclidean distance is used:

$$D = \sqrt{(x_1 - w_1)^2 + (x_2 - w_2)^2}$$

42.2.4 Weight Update Rule

$$W = W + \alpha(X - W)$$

where:

- W = weight vector
- X = input vector
- α = learning rate

42.2.5 Numerical Problem

Given input vectors:

$$A = (1, 2), \quad B = (2, 3), \quad C = (5, 6)$$

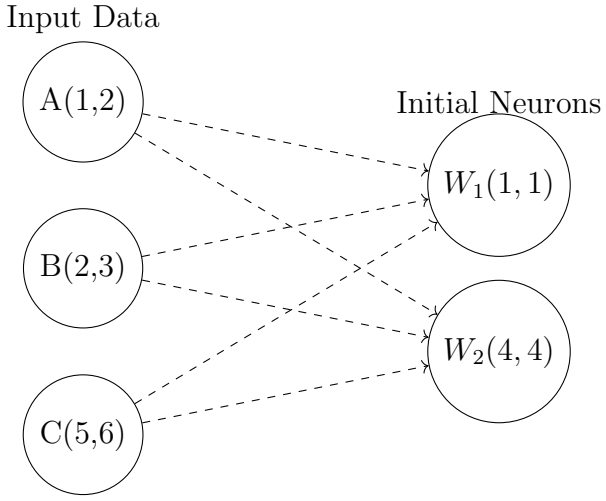
Initial weight vectors are randomly initialized as:

$$W_1 = (1, 1), \quad W_2 = (4, 4)$$

Learning rate:

$$\alpha = 0.5$$

42.2.6 Initial SOM Structure (Before Training)



42.3 Iteration 1

42.3.1 Input A = (1,2)

We compute Euclidean distance between input and each neuron:

$$D_1 = \sqrt{(1-1)^2 + (2-1)^2} = \sqrt{0+1} = 1$$

$$D_2 = \sqrt{(1-4)^2 + (2-4)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.6$$

Since $D_1 < D_2$, the Best Matching Unit (BMU) is W_1. Weight update rule:

$$W = W + \alpha(X - W)$$

Update W_1:

$$W_1 = (1,1) + 0.5[(1,2) - (1,1)]$$
$$W_1 = (1,1) + 0.5(0,1)$$
$$W_1 = (1,1.5)$$

42.3.2 Input B = (2,3)

Distances:

$$D_1 = \sqrt{(2-1)^2 + (3-1.5)^2} = \sqrt{1+2.25} = \sqrt{3.25} \approx 1.8$$
$$D_2 = \sqrt{(2-4)^2 + (3-4)^2} = \sqrt{4+1} = \sqrt{5} \approx 2.23$$

Since $D_1 < D_2$, BMU = W_1. Update:

$$W_1 = (1,1.5) + 0.5[(2,3) - (1,1.5)]$$
$$W_1 = (1,1.5) + 0.5(1,1.5)$$
$$W_1 = (1.5,2.25)$$

42.3.3 Input C = (5,6)

Distances:

$$D_1 = \sqrt{(5-1.5)^2 + (6-2.25)^2} = \sqrt{12.25+14.06} = \sqrt{26.31} \approx 5.1$$
$$D_2 = \sqrt{(5-4)^2 + (6-4)^2} = \sqrt{1+4} = \sqrt{5} \approx 2.23$$

Since $D_2 < D_1$, BMU = W_2. Update:

$$W_2 = (4,4) + 0.5[(5,6) - (4,4)]$$
$$W_2 = (4,4) + 0.5(1,2)$$
$$W_2 = (4.5,5)$$

42.4 Iteration 2

42.4.1 Input A = (1,2)

Distances:

$$D_1 = \sqrt{(1-1.5)^2 + (2-2.25)^2} = \sqrt{0.25+0.0625} = \sqrt{0.3125} \approx 0.56$$
$$D_2 = \sqrt{(1-4.5)^2 + (2-5)^2} = \sqrt{12.25+9} = \sqrt{21.25} \approx 4.6$$

BMU = W_1. Update:

$$W_1 = (1.5,2.25) + 0.5[(1,2) - (1.5,2.25)]$$
$$W_1 = (1.5,2.25) + 0.5(-0.5,-0.25)$$
$$W_1 = (1.25,2.125)$$

42.4.2 Input B = (2,3)

Distances:

$$D_1 = \sqrt{(2-1.25)^2 + (3-2.125)^2} = \sqrt{0.5625 + 0.7656} = \sqrt{1.3281} \approx 1.15$$

$$D_2 = \sqrt{(2-4.5)^2 + (3-5)^2} = \sqrt{6.25 + 4} = \sqrt{10.25} \approx 3.2$$

BMU = W_1. Update:

$$W_1 = (1.25, 2.125) + 0.5[(2,3) - (1.25, 2.125)]$$

$$W_1 = (1.25, 2.125) + 0.5(0.75, 0.875)$$

$$W_1 = (1.625, 2.5625)$$

42.4.3 Input C = (5,6)

Distances:

$$D_1 = \sqrt{(5-1.625)^2 + (6-2.5625)^2} = \sqrt{11.39 + 11.82} = \sqrt{23.21} \approx 4.82$$

$$D_2 = \sqrt{(5-4.5)^2 + (6-5)^2} = \sqrt{0.25 + 1} = \sqrt{1.25} \approx 1.12$$

BMU = W_2. Update:

$$W_2 = (4.5, 5) + 0.5[(5,6) - (4.5, 5)]$$

$$W_2 = (4.5, 5) + 0.5(0.5, 1)$$

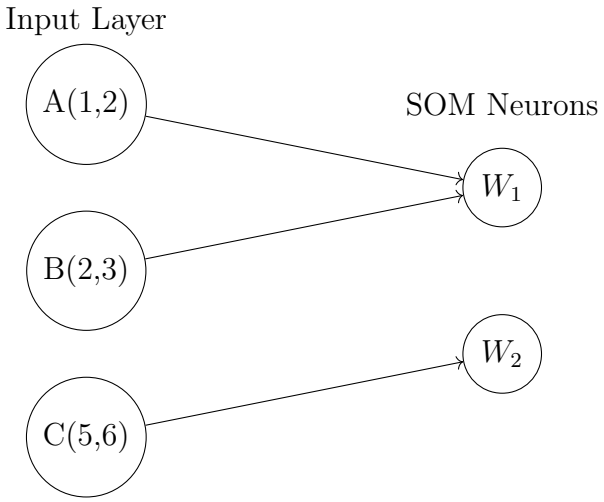
$$W_2 = (4.75, 5.5)$$

42.5 Final Weights After 2 Iterations

$$W_1 = (1.625, 2.5625)$$

$$W_2 = (4.75, 5.5)$$

42.5.1 SOM Neuron Structure (Vertical Representation)



42.5.2 Interpretation

- W_1 represents cluster of smaller values (A, B)
- W_2 represents cluster of larger values (C)
- Weights gradually move toward input patterns
- SOM performs clustering without labeled data

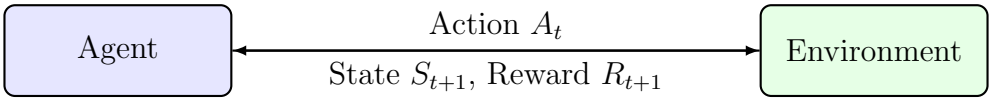
43 Lecture 13: Reinforcement Learning (RL)

43.1 Introduction

Reinforcement Learning (RL) is a branch of Machine Learning in which an agent learns by interacting with an environment. The agent performs actions, receives rewards or penalties, and gradually learns the optimal behavior that maximizes long-term cumulative reward.

Unlike supervised learning, RL does not require labeled input-output pairs. Learning occurs through trial and error.

43.2 Basic RL Interaction



43.3 Elements of Reinforcement Learning

43.3.1 Agent

The agent is the learner or decision maker that interacts with the environment.

Examples:

- Robot
- Self-driving car
- Game-playing AI
- Trading bot

Mathematically:

$$A_t$$

where:

- A_t = action selected at time t

43.3.2 Environment

The environment is everything outside the agent.

It receives the agent's actions and responds with:

- new state
- reward

Examples:

- Chess board
- Road network
- Video game
- Stock market

43.3.3 State

A state represents the current condition of the environment.

Examples:

- Position of robot
- Current traffic condition
- Current game frame

Mathematically:

$$S_t$$

where:

- S_t = state at time t

43.3.4 Action

An action is a decision taken by the agent.

Examples:

- Move left/right
- Accelerate/brake
- Buy/sell stock

Mathematically:

$$A_t \in \mathcal{A}$$

where:

- $\mathcal{A}$ = action space

43.3.5 Reward

A reward is feedback received from the environment after performing an action.

Examples:

- +100 for winning
- -10 for collision
- -1 for unnecessary movement

Mathematically:

$$R_{t+1}$$

where:

- R_{t+1} = reward received after action

43.4 Goal of RL

The objective of RL is to maximize cumulative future reward.

The total future reward is called return.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

where:

- G_t = return
- γ = discount factor

43.5 Discount Factor

The discount factor controls the importance of future rewards.

$$0 \leq \gamma \leq 1$$

- $\gamma = 0$: only immediate reward matters
- $\gamma \approx 1$: future rewards are important

43.6 Application of Deep Learning in RL

Deep Learning is combined with RL to handle large and complex state spaces.

This combination is called:

Deep Reinforcement Learning

Applications:

- Self-driving cars
- Robotics
- Healthcare
- Game AI
- Finance

43.7 Deep Q-Network (DQN)

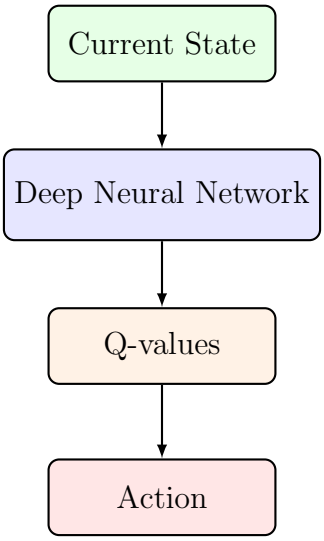
Deep Q-Networks use neural networks to estimate Q-values.

$$Q(s, a)$$

where:

- $Q(s, a)$ = expected future reward for taking action a in state s

43.8 DQN Architecture



44 Markov Decision Process (MDP)

44.1 Introduction

A Markov Decision Process (MDP) is the mathematical framework used in Reinforcement Learning.

An MDP is represented as:

$$(S, A, P, R, \gamma)$$

where:

Symbol	Meaning
S	Set of states
A	Set of actions
P	Transition probabilities
R	Reward function
γ	Discount factor

44.2 Transition Probabilities

Transition probability defines the probability of moving from one state to another after taking an action.

$$P(s'|s, a)$$

where:

- s = current state
- a = action
- s' = next state

44.3 Markov Property

The future depends only on:

- current state
- current action

Mathematically:

$$P(S_{t+1}|S_t, A_t)$$

Past history does not directly affect future states.

44.4 Policy

A policy defines the behavior of the agent.

It specifies which action the agent should take in a particular state.

44.4.1 Deterministic Policy

A deterministic policy always selects the same action for a given state.

$$a = \pi(s)$$

where:

- s = current state
- a = selected action
- π = policy

Example of Deterministic Policy Consider a self-driving car approaching a traffic signal.

State	Action Selected
Red Signal	Stop
Green Signal	Move Forward
Yellow Signal	Slow Down

Here:

- Every time the car observes a red signal, it always chooses "Stop".
- The same state always produces the same action.

Thus, the policy is deterministic.

44.4.2 Stochastic Policy

A stochastic policy selects actions according to probabilities.

$$\pi(a|s)$$

where:

- $\pi(a|s)$ = probability of selecting action a in state s

Example of Stochastic Policy Consider a robot navigating a maze.
At state S_1, the robot may choose different actions with different probabilities.

Action	Probability
Move Left	0.7
Move Right	0.2
Move Up	0.1

This means:

- 70% probability of moving left
- 20% probability of moving right
- 10% probability of moving up

The robot does not always choose the same action.
Thus, the policy is stochastic.

44.5 Value Function

The value function measures how good a state is.

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t|S_t = s]$$

where:

- $V^{\pi}(s)$ = expected future reward from state s

44.6 Q-Value Function

The Q-value function measures how good an action is in a state.

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t|S_t = s, A_t = a]$$

44.7 Bellman Equation for Value Function

$$V(s) = \sum_{s'} P(s'|s, a) \left[R + \gamma V(s')\right]$$

44.8 Bellman Equation for Q-Function

$$Q(s,a) = R + \gamma \sum_{s'} P(s'|s,a)V(s')$$

44.9 Difference Between $V(s)$ and $Q(s,a)$

Value Function $V(s)$	Q-Value Function $Q(s,a)$
Measures goodness of state	Measures goodness of action-state pair
Depends only on state	Depends on state and action
Used for state evaluation	Used for action selection

44.10 Exploration vs Exploitation

Exploration	Exploitation
Trying new actions	Using best known action
Discovers new knowledge	Maximizes reward
Risky	Safe

44.11 ϵ-Greedy Strategy

The ϵ-greedy strategy balances exploration and exploitation.

- With probability $(1-\epsilon)$: choose best action
- With probability ϵ: choose random action

Example:

$$\epsilon = 0.2$$

Then:

- 80% exploitation
- 20% exploration

44.12 Q-Learning (Model-Free Reinforcement Learning)

Q-learning is a model-free, off-policy Reinforcement Learning algorithm that learns the optimal action-value function $Q^*(s,a)$ without requiring the transition model $P(s'|s,a)$.

It directly learns from interaction with the environment.

44.12.1 Q-Learning Update Rule

The core update rule of Q-learning is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big]$$

where:

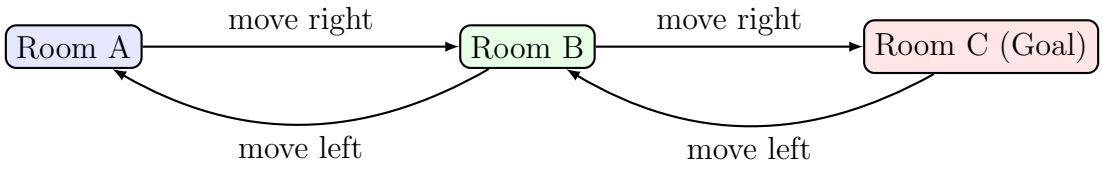
- α = learning rate
- γ = discount factor
- $\max_{a'} Q(S_{t+1}, a')$ = best future reward estimate
- R_{t+1} = immediate reward

44.13 Numerical Example: Robot in House (Room Navigation)

44.13.1 Scenario

A robot is inside a house with 3 rooms:

- Room A (Start)
- Room B
- Room C (Goal)



44.13.2 Reward Structure

Action Outcome	Reward
Reach Goal (Room C)	+10
Each step	-1
Invalid move	-5

44.13.3 Initial Q-Table

We initialize all Q-values to zero.

State	Left	Right	Stay
A	0	0	0
B	0	0	0
C	0	0	0

44.13.4 Episode 1: Learning Step-by-Step

Assume:

$$\alpha = 0.5, \quad \gamma = 0.9$$

Step 1: A → B

- Current state: A
- Action: Right
- Reward: -1
- Next state: B

Update:

$$Q(A, Right) = 0 + 0.5[-1 + 0.9 \cdot \max Q(B, \cdot) - 0]$$

Since all Q(B,·)=0:

$$Q(A, Right) = -0.5$$

Step 2: B → C

- Current state: B
- Action: Right
- Reward: +10
- Next state: C (Goal)

$$Q(B, Right) = 0 + 0.5[10 + 0 - 0]$$

$$Q(B, Right) = 5$$

44.13.5 Updated Q-Table after Episode 1

State	Left	Right	Stay
A	0	-0.5	0
B	0	5	0
C	0	0	0

44.13.6 Key Insight

- The agent learns that moving from B to C is highly valuable.
- The value of A $\rightarrow$ B becomes negative due to step cost.
- Over time, repeated updates converge to optimal policy.

44.14 Numerical Example: Grid World Navigation (2D Q-Learning)

44.14.1 Scenario

Consider a robot navigating a 3×3 grid house.

The robot starts at S and must reach the goal G.

Some cells are safe, and some give penalties.

S		
	-5	
		G

44.14.2 Grid Description

State	Meaning	Reward
S	Start (top-left)	0
I	Intermediate states	-1
X	Obstacle (center)	-5
G	Goal (bottom-right)	+10

44.14.3 Actions

The robot can move in 4 directions:

$$A = \{Up, Down, Left, Right\}$$

Invalid moves keep the agent in the same state with penalty.

44.14.4 Learning Parameters

$$\alpha = 0.5, \quad \gamma = 0.9$$

All Q-values are initialized to 0.

44.14.5 Episode 1: Step-by-Step Learning

We track one full episode from start to goal.

Step 1: S → (move Right)

- State: S
- Action: Right
- Reward: -1
- Next state: I

$$Q(S, Right) = 0 + 0.5[-1 + 0.9 \cdot \max Q(I, a) - 0]$$

Since all Q-values are 0:

$$Q(S, Right) = -0.5$$

Step 2: I → (move Down)

- State: I
- Action: Down
- Reward: -1
- Next state: X (obstacle)

$$Q(I, Down) = 0 + 0.5[-1 + 0.9 \cdot \max Q(X, a)]$$

$$Q(I, Down) = -0.5$$

Step 3: I → (avoid obstacle, move Right)

- State: I
- Action: Right
- Reward: -1
- Next state: I2 (safe cell)

$$Q(I, Right) = 0 + 0.5[-1 + 0.9 \cdot 0]$$

$$Q(I, Right) = -0.5$$

Step 4: I2 → G

- State: I2
- Action: Right

- Reward: +10
- Next state: Goal

$$Q(I2, Right) = 0 + 0.5[10 + 0]$$

$$Q(I2, Right) = 5$$

44.14.6 Updated Q-Table (After Episode 1)

State	Up	Down	Left	Right
S	0	0	0	-0.5
I	0	-0.5	0	-0.5
I2	0	0	0	5
G	0	0	0	0
X	0	0	0	0

44.15 Relation to DQN

Q-learning uses a table:

$$Q(s, a)$$

When the state space becomes large, we replace the table with a neural network:

$$Q(s, a; \theta)$$

This leads to Deep Q-Networks (DQN).